LOC8

LOC8: AN IOT-BASED SMART PARCEL MANAGEMENT SYSTEM WITH AGENTIC AI NOTIFICATION AND RFID INTEGRATION

FAEZ YAHIZAN

Table Of Contents

CHAPTER 1: INTRODUCTION

1.1 Background of Study

The rapid growth of e-commerce has led to a significant surge in parcel deliveries within university campuses. Conventional parcel management systems in residential colleges often rely on manual logging which is prone to human error and results in long waiting times during peak hours.

Loc8 is an IoT-based innovation designed to streamline this process. By integrating ESP32 microcontrollers and RFID technology, the system automates the identification and location tracking of parcels ensuring a more efficient and secure retrieval experience for students.

1.2 Vision and Mission

Vision: To become the standard for automated smart-campus logistics, fostering a seamless and technologically advanced environment for student services.

Mission: To provide a reliable, low-cost, and user-friendly automated parcel tracking system that reduces operational bottlenecks and enhances data integrity through smart hardware integration.

1.3 Market Location

The primary implementation site for the **Loc8** system is **Gerai Tanjung, Universiti Teknologi MARA (UiTM) Jasin** as shown in Figure 1.1 & Figure 1.2.

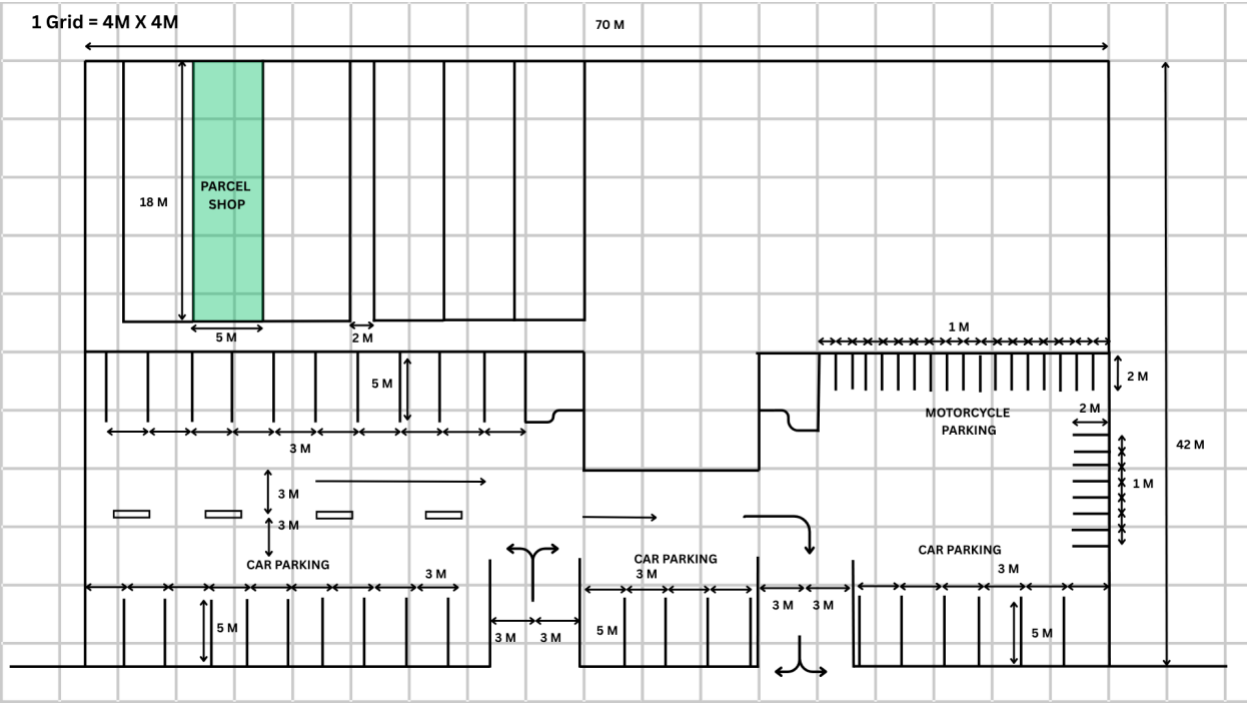


Figure 1.1 : Top View Of Pusat Persatuan Pelajar, UiTM Jasin

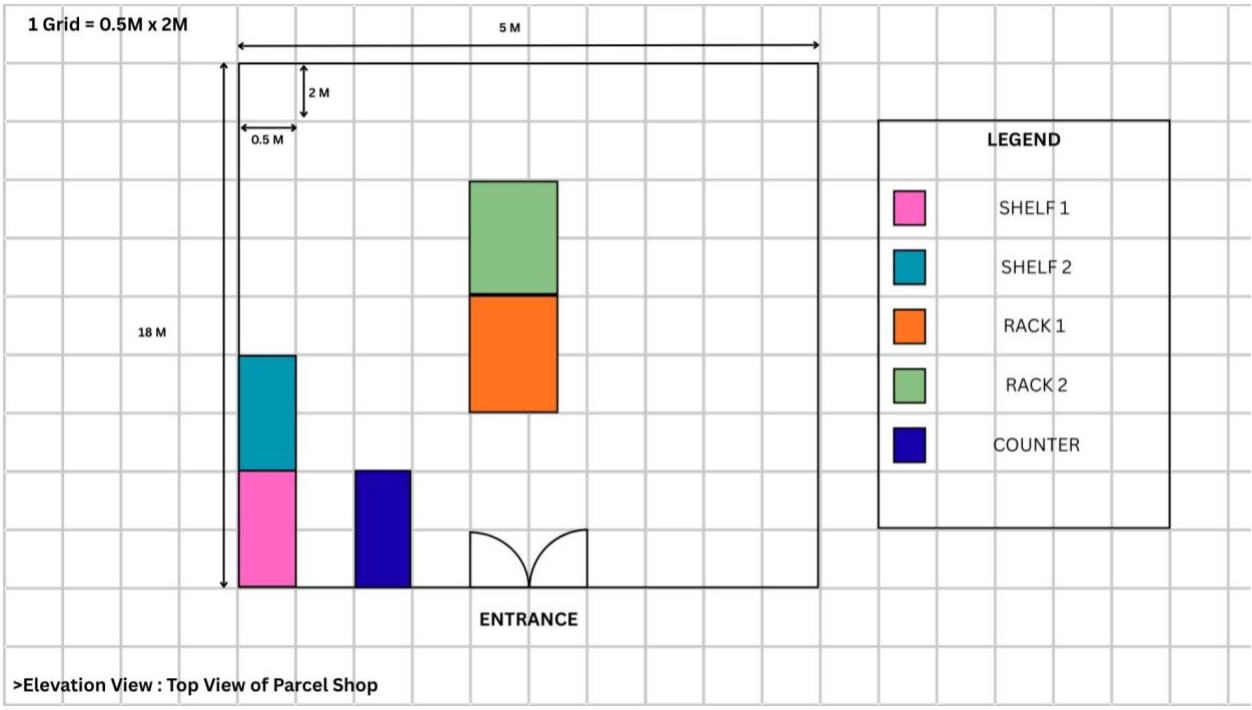


Figure 1.2 : Top View Of Parcel Shop (Gerai Tanjung)

1.4 Potential Clients

The primary target for this innovation is the **Shop Vendor** who is already managing the shop.

1.4.1 The Shop Vendor

Shop vendor who have leased the centralized space from the university and are looking for technological solutions to manage high-volume turnover without increasing staff costs.

1.4.2 The Beneficiaries

1. The University (Stakeholders)

Interested in seeing their existing facilities modernized to improve student welfare and campus image.

2. The Students

End-users who will experience a faster and more professional retrieval experience.

1.5 Logo

Figure 1.3 : Logo

1.6 Project Organization and Contribution

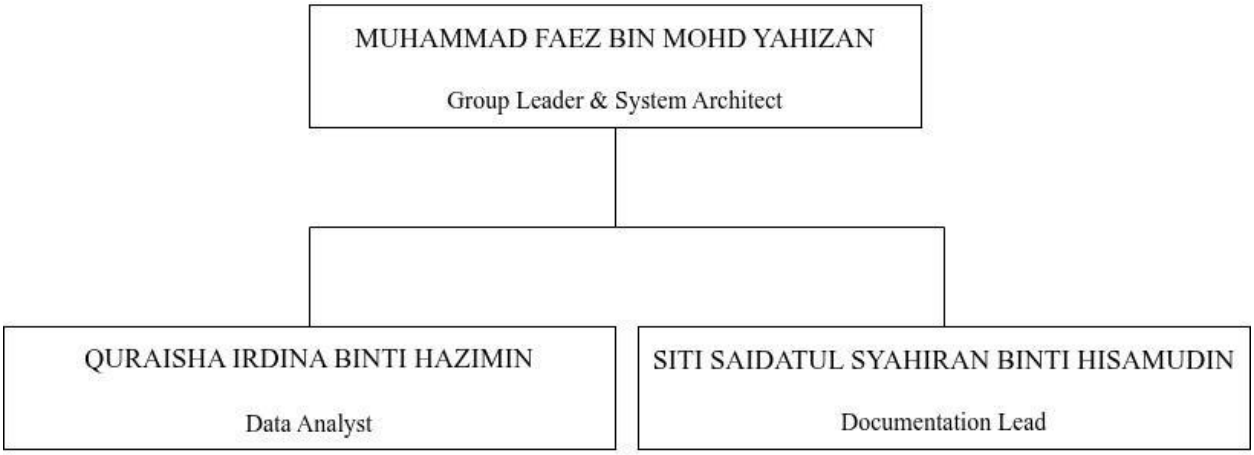


Figure 1.4 : Group Organizational Chart

1.6.1 Group Leader & System Architect

Contribution : IoT Infrastructure & Backend Development

- ➢ ESP32 & sensor hardware integration.
- ➢ Backend API development.
- ➢ System performance optimization.
- ➢ System performance optimization.

1.6.2 Data Analyst

Contribution : Data Governance & System Automation

- ➢ Agentic AI/Telegram automation.
- ➢ Database schema management.
- ➢ Rigorous system test-case validation.

1.6.3 Documentation Lead

Contribution : UX Design & Technical Documentation

- ➢ Formal academic documentation.
- ➢ UI/UX dashboard optimization.
- ➢ Visual synthesis of system schematics.

1.7 Problem Statement

Gerai Tanjung at UiTM Jasin serves as a vital centralized parcel hub for the campus community. However, the current operational model, managed by the shop vendor, faces several critical challenges that hinder its efficiency.

1.7.1 Inefficient Manual Record-Keeping

Staff manually write recipient names and parcel numbers to track locations. This process is time-consuming and creates bottlenecks for students to check their names and staff to write during peak hours.

1.7.2 Data Redundancy and Human Error

Handwritten logs are susceptible to illegible handwriting or incorrect shelf labeling as shown in Figure 1.5. Without a digital search function, items physically present but missing from records frequently occur.

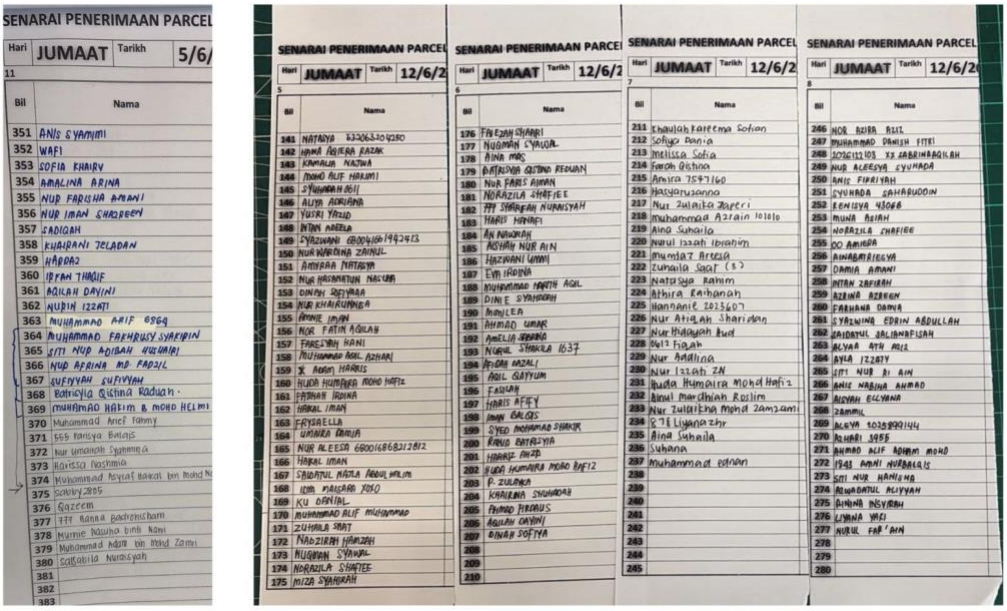


Figure 1.5 : Handwritten Logs

1.7.3 Lack of Digital Integration (Digitization Gap)

There is no link between physical parcels and a searchable database. **Loc8** bridges this "digitization gap" by linking RFID-tagged parcels to Google Sheets for instant data retrieval.

1.7.4 Retrieval Inefficiency for Staff

Currently, staff spend excessive time searching for parcels due to the lack of a structured digital location system. Without an instant search function, staff must manually scan shelves, leading to slow service delivery and increased student wait times.

1.8 Objectives

The main objective is to modernize the manual parcel management system at Gerai Tanjung:

1. To design and implement a cloud-based recording system using Google Sheets to replace manual logbooks.
2. To develop a hardware prototype using the ESP32 and MFRC522 RFID sensor for instant info retrieval.
3. To reduce human errors by automating the matching process.
4. To integrate Google Sheets with a web-based dashboard for real-time parcel data visualization.

CHAPTER 2: IOT SYSTEM JUSTIFICATION

2.1 System Definition and Boundary

2.1.1 Operational Scope

The implementation and testing of the system are strictly limited to **Gerai Tanjung at UiTM Jasin**. The system is tailored to the specific shelf layout and parcel volume of this centralized hub and may require recalibration for different environments.

2.1.2 Connectivity Requirements

The system is designed to be relying on **any available Wi-Fi infrastructure** for cloud synchronization. Stable internet connectivity is mandatory for the ESP32 to communicate with the Google Sheets database via HTTPS requests.

2.1.3 Data Capacity and Storage

Loc8 utilizes **Google Sheets** as its primary cloud database. While cost-effective and accessible, the system is subject to Google's API quotas and the maximum cell limit of a single spreadsheet. It is designed for medium-scale campus operations rather than high-frequency industrial logistics.

2.1.4 Identification Technology

The system is bounded by the frequency range of the **MFRC522 RFID module (13.56 MHz)**. It is designed to read standard passive tags and student ID cards that operate within this specific frequency, and it will not detect barcodes or UHF RFID tags.

2.2 IOT Component Requirements

No	Component	Purpose
1	ESP32 NodeMCU	Central processing unit and Wi-Fi gateway.
2	MFRC522 RFID Module	Passive RFID tag identification at 13.56 MHz.
3	LCD 2004 (I2C)	Visual interface for status and location display.
4	Piezo Buzzer	Auditory notification for success/error feedback.
5	LED Indicators	Visual status indicators for operational modes.
6	Tactile Push Button	Serves as a mode-toggle switch (Register/Verify mode).
7	Jumper Wires	Establish signal connectivity between sensors and ESP32.

2.3 Software & Application Requirements

2.3.1 Firmware and Microcontroller Development

No	Development Environment	Purpose
1	Arduino IDE	The primary Integrated Development Environment (IDE) used for writing, compiling, and uploading the firmware to the ESP32 microcontroller.
2	C++	The core programming language utilized to manage hardware abstraction, sensor interfacing, and Wi-Fi communication protocols.

2.3.2 External Libraries

No	External Libraries	Purpose
1	MFRC522.h	Facilitates SPI communication with the RFID reader.
2	LiquidCrystal_I2C.h	Enables I2C-based communication for the 20x4 LCD module.
3	ArduinoJson.h	Essential for parsing JSON payloads received from the cloud database.
4	WiFi.h & HTTPClient.h	Manages the ESP32 network stack and HTTPS connectivity.

2.3.3 Cloud Infrastructure and Backend

No	External Libraries	Purpose
1	Google Sheets API	Serves as the primary relational database for storing parcel records, student information, and transaction logs.
2	Google Apps Script	A server-side JavaScript environment that functions as the system's "brains." It executes the logic for parcel registration, status verification, and automated notification triggers.
3	Autonomous Agentic AI Logic	An event-driven script triggered nightly to monitor parcel statuses (items unclaimed for >3 days) and autonomously initiate notification sequences via the Telegram Bot API.

2.3.4 Development and Deployment Tools

The Telegram Bot API serves as the communication gateway for the Autonomous Notification System, facilitating the secure and instantaneous delivery of parcel status updates to students.

2.4 Technology Justification RFID vs. QR Code

In the development of the **Loc8** system, a comparative analysis was conducted between Radio Frequency Identification (RFID) and Quick Response (QR) codes. While QR codes are cost-effective, RFID was selected as the primary identification technology for the Gerai Tanjung Parcel Hub due to several strategic advantages.

2.4.1 Non-Line-of-Sight (NLoS) Capability

RFID technology is superior to QR codes because it utilizes radio waves that can penetrate non-metallic materials like plastic packaging and cardboard, eliminating the need for a direct line of sight. This allows staff to scan parcels near-instantly without having to manually rotate or re-orient heavy and awkward boxes to locate a scanning sticker, even if the tag is hidden beneath layers of packaging.

2.4.2 Durability in Operational Conditions

RFID technology is chosen over QR codes due to its superior durability against rough handling, friction, and environmental factors like rain during transit. Unlike QR codes printed on thermal paper which are prone to smudging, tearing, or fading. RFID tags utilize an embedded internal chip and antenna that remain functional even when the outer packaging is damaged or dirty, thereby ensuring consistent data integrity for the parcel tracking system.

2.4.3 Operational Speed and Efficiency

RFID technology is implemented to eliminate operational bottlenecks, as it allows for near-instantaneous data capture upon proximity to the reader, unlike QR scanning which requires a time-consuming "focus and capture" process sensitive to lighting and distance. This rapid scanning capability enables staff at Gerai Tanjung to process multiple parcels quickly during peak hours, significantly increasing the turnover rate and service efficiency compared to traditional visual scanning methods.

2.4.4 Integration with Existing Infrastructure

The Loc8 system leverages the existing RFID/NFC technology embedded in UiTM student identification cards, eliminating the need for students to generate or present digital QR codes via smartphones. This integration ensures a seamless verification process through a simple physical tap, providing a fail-safe solution that remains functional even in situations where a student's mobile device has a depleted battery or lacks internet connectivity.

2.5 Technology Justification RFID vs. NFC

In configuring the wireless data capture architecture for the Loc8 system, a technical evaluation was conducted between standard Radio Frequency Identification (RFID) and Near Field Communication (NFC). Although NFC is technically a specialized branch of High-Frequency RFID (13.56 MHz), standard RFID was selected as the core tracking mechanism for the Gerai Tanjung Parcel Hub due to its overwhelming operational superiority in logistics and asset management.

2.5.1 Extended Read Range and Spatial Flexibility

Standard Ultra-High Frequency (UHF) or standard RFID configurations are designed for broader proximity data capture, making them highly efficient for scanning and locating packages within the dynamic storage layout of the Gerai Tanjung Parcel Hub. Conversely, NFC is restricted to an ultra-short-range transmission zone of less than 4 cm.

.5.2 Optimization for High-Volume Asset Tracking

NFC is fundamentally designed for single-target authentication (one reader to one tag at a close distance), which introduces severe operational bottlenecks when handling a high volume of inventory. In contrast, RFID technology is built for industrial logistics, supporting anti-collision algorithms that enable bulk scanning capabilities. This allows the Loc8 system to efficiently process incoming parcel batches, register shelf allocations, and manage inventory without forcing the operator to scan every single package individually at point-blank range.

CHAPTER 3: SYSTEM DESIGN

3.0 Introduction to System Design

This chapter outlines the logical and physical architecture of the **Loc8** system. It provides a visual guide through flowcharts and circuit diagrams.

3.1 Requirement Analysis

Before designing the Loc8 system, a preliminary study was conducted via a **User Feedback Google Form** to identify the specific challenges faced by students at Gerai Tanjung. This phase is crucial to ensure the proposed system addresses the actual "pain points" of the current manual process.

Google Form Link : https://forms.gle/Ko3bi61vT6MwoTct6

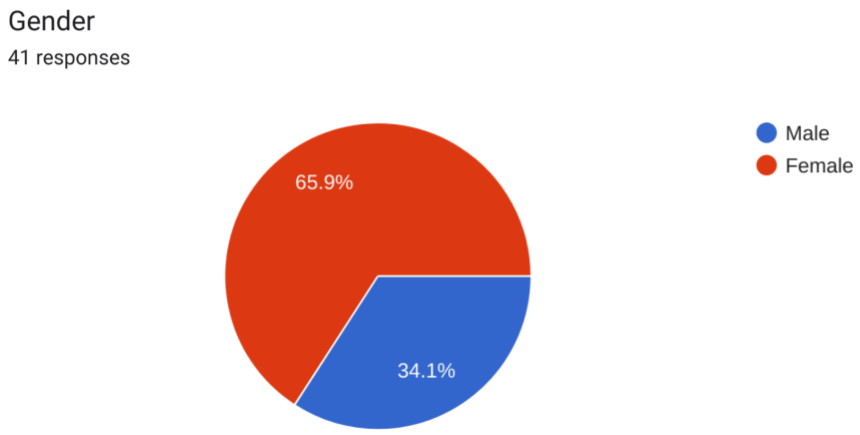


Figure 3.1 : Gender Distribution of Respondents

3.1.1 Existing System Workflow

The questions regarding the existing system workflow are presented in Figure A.1 (Appendix A). It displays the survey questions designed to evaluate the current manual verification bottlenecks.

1: Strongly Disagree to 5: Strongly Agree.

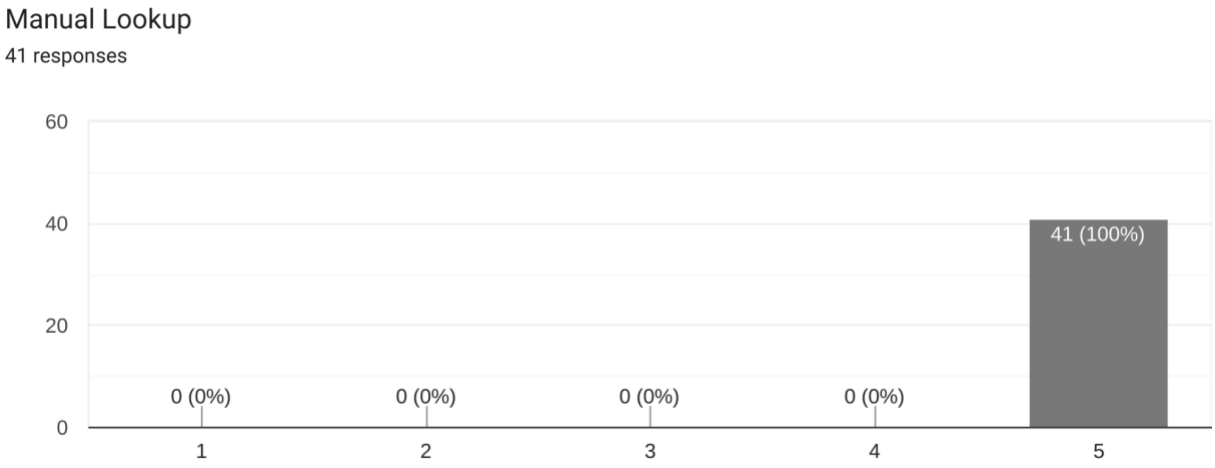


Figure 3.2 : Respondents' Agreement on Manual Lookup Usage

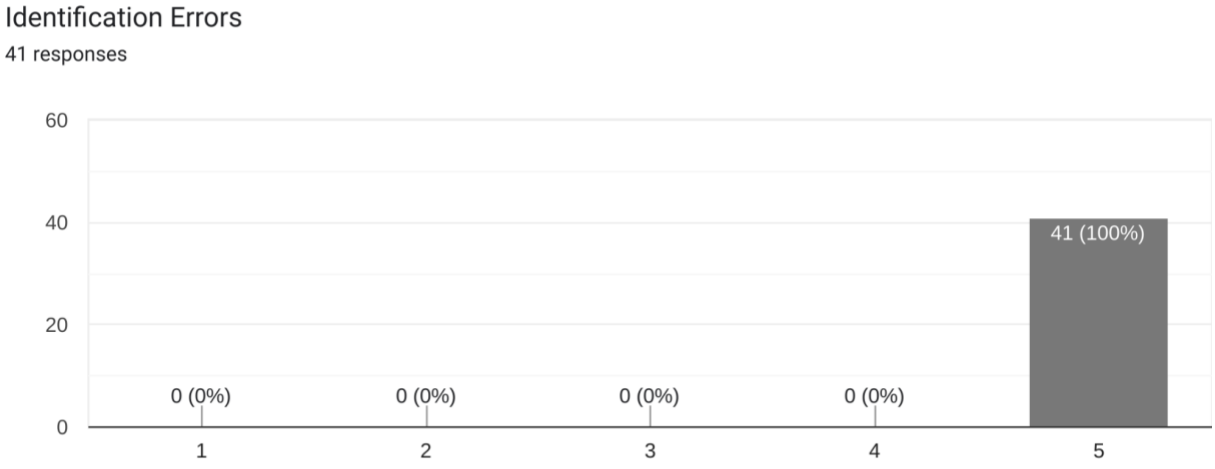


Figure 3.3 : Respondents' Agreement on Identification Errors Made

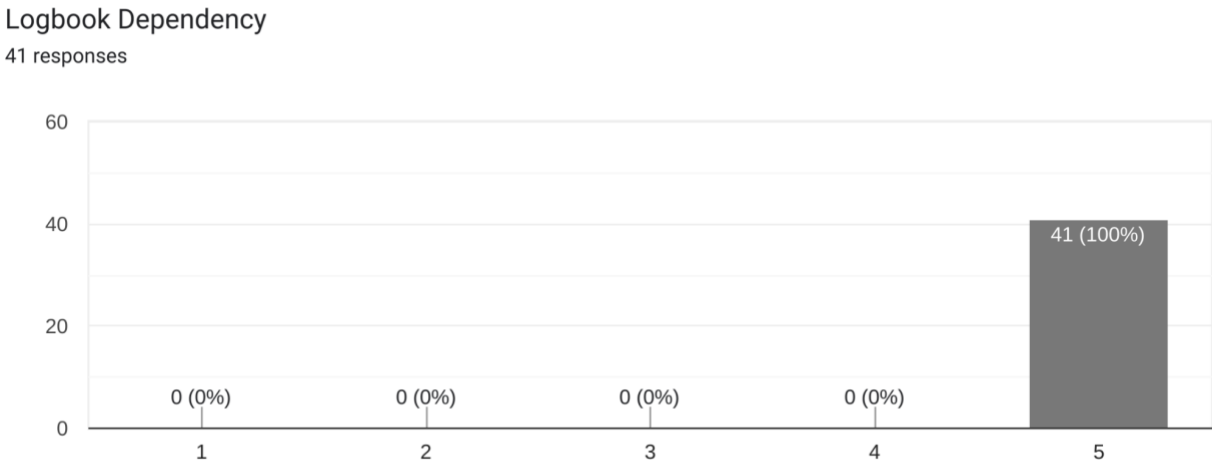


Figure 3.4 : Respondents' Agreement on Logbook Dependency

3.1.2 Proposed System Workflow

The questions regarding the proposed system workflow are presented in Figure A.2 (Appendix A). This section evaluates user agreement on the proposed solution's core benefits.

1: Strongly Disagree to 5: Strongly Agree.

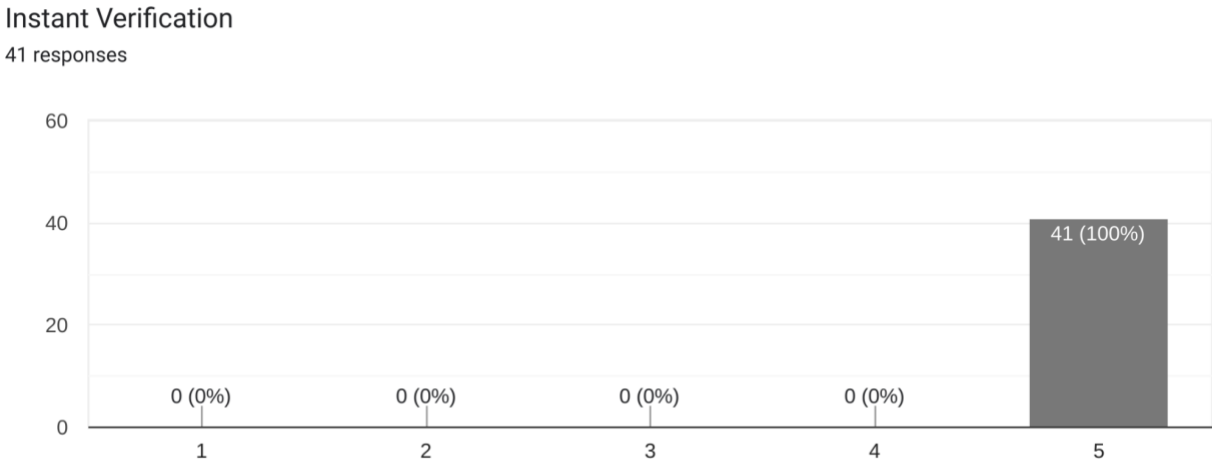


Figure 3.5 : Respondents' Agreement on Instant Verification

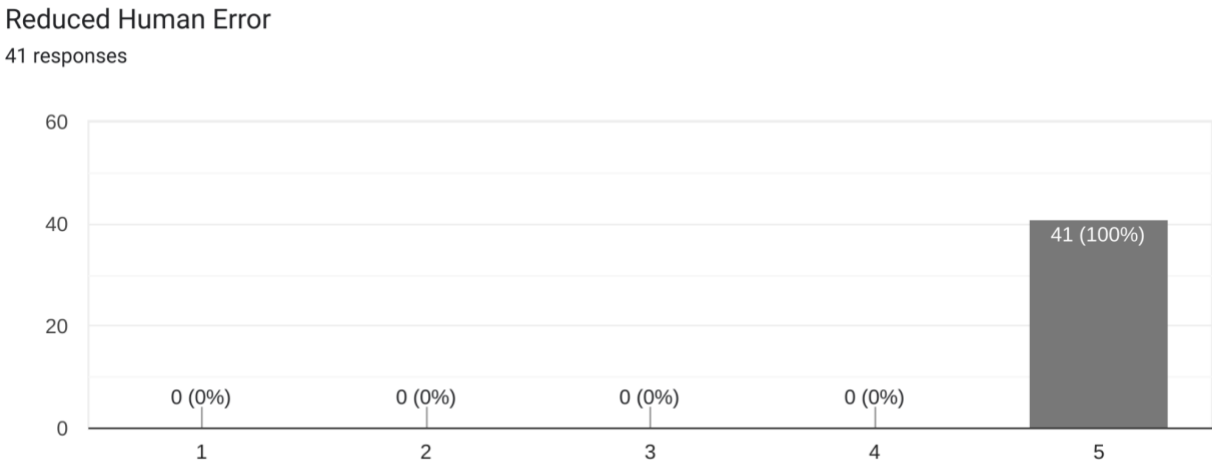


Figure 3.6 : Respondents' Agreement on Reduced Human Error

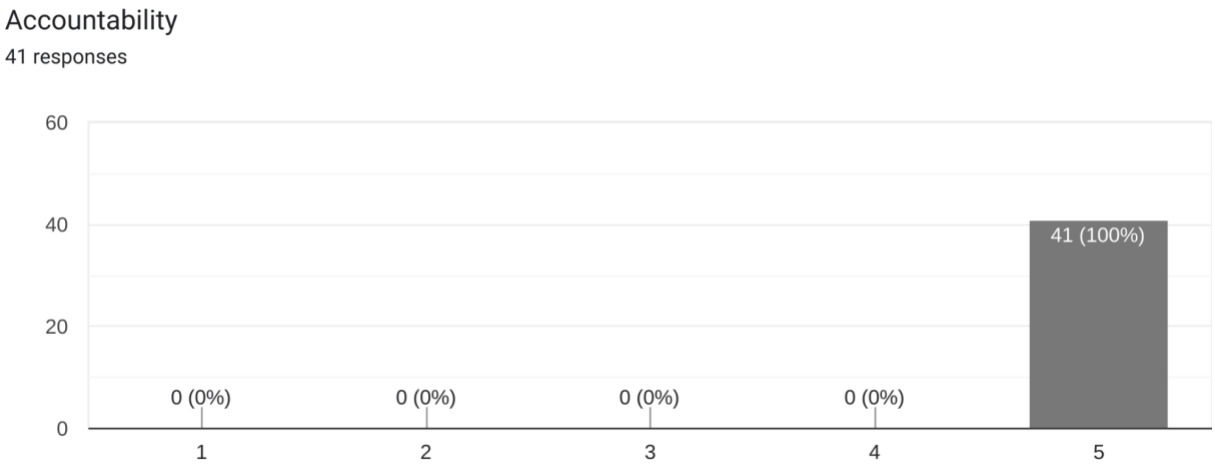


Figure 3.7 : Respondents' Agreement on Accountability

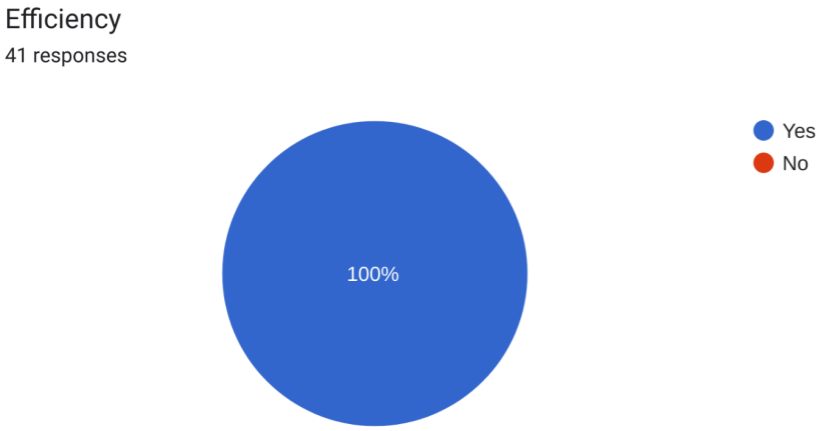


Figure 3.8 : Respondents' Agreement on Efficiency

3.1.3 User Feedback

The questions regarding user feedback are presented in Figure A.3 (Appendix A). The final section of the questionnaire focuses on the comparative efficiency of the proposed workflow and the economic feasibility of the system.

Based on the feedback collected from the campus community, students were asked to identify the improvements they would like to see in the current manual parcel system as shown in Figure 3.9

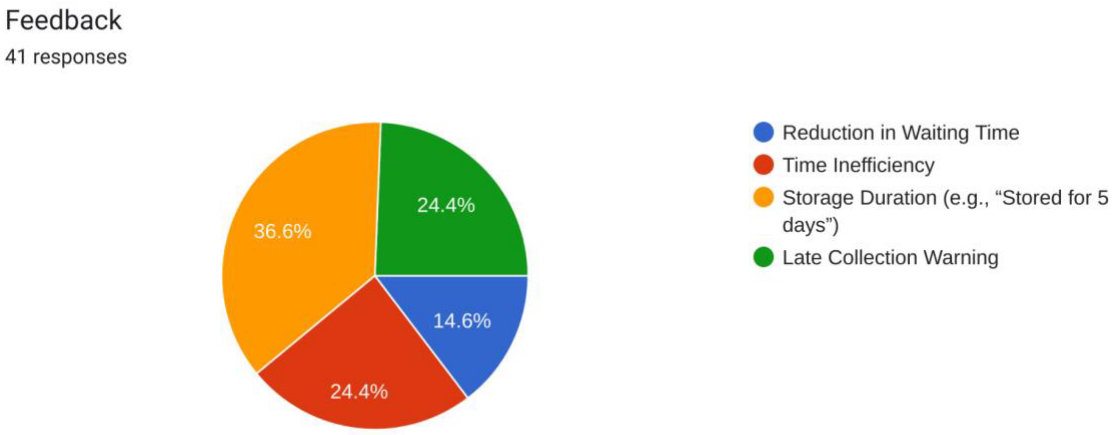


Figure 3.9 : Suggested Improvements for the Manual Parcel System

The survey also inquired whether students would accept a price increase from RM1.00 to RM1.50 for the RFID sticker as shown in Figure 3.10.

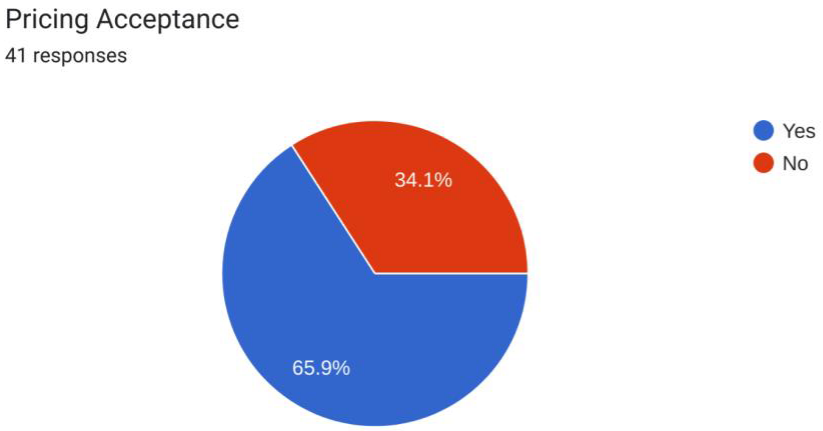


Figure 3.10 : Respondents' Acceptance of RFID Sticker Price Increase

Market research was conducted to identify ways to minimize costs as high service fees may discourage students from utilizing the parcel service. Although the current market rate for a single RFID tag is approximately RM1.00 (bringing the total cost to RM2.00 including service), our research indicates that the cost can be significantly reduced.

For instance, sourcing from wholesale platforms like Alibaba allows the purchase of RFID tags at approximately RM0.2841 per unit provided when **a minimum order of 1,000 units to 9,999 units** is met as shown in Figure 3.11. The total cost would be RM 1.50.

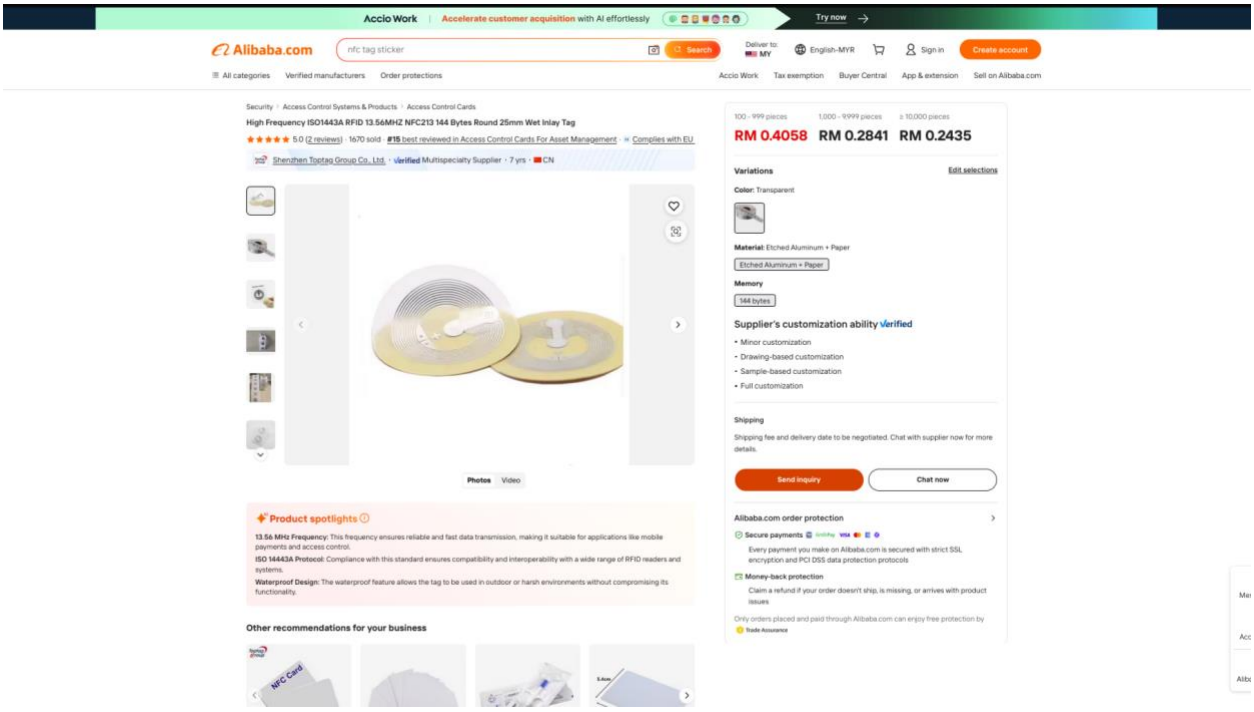


Figure 3.11 : RFID Tag Sticker in Alibaba Website (*High Frequency ISO1443A RFID 13.56MHZ NFC213 144 Bytes Round 25mm Wet Inlay Tag*, 2022)

3.2 Control System: Legacy vs. Proposed Workflow

3.2.1 Legacy Manual Workflow (Existing)

➢ Flowchart 1: Student Side

The end-to-end process mapping for students is illustrated in Figure A.4 (Appendix A).

➢ Flowchart 2: Vendor Side

The operational workflow for the shop vendors is presented in Figure A.5 (Appendix A).

3.2.2 Proposed Loc8 Digitalized Workflow (Proposed)

➢ Flowchart 1: Student Side

By utilizing automated card-scanning authentication, the system eliminates traditional bottlenecks, allowing students to securely verify and retrieve their parcels within seconds. The complete step-by-step process flow is documented in A.6 (Appendix A).

➢ Flowchart 2: Vendor Side

Vendors can easily search, track, and update data directly through a seamless digital interface. Scanning an RFID tag automatically fetches and displays the relevant information instantly, allowing for rapid digital editing and real-time database updates. This operational workflow is fully detailed in A.7 (Appendix A).

➢ Flowchart 3: System

Detailed system flowchart is shown in Figure A.8 (Appendix A).

➢ Flowchart 4: Agentic AI

Detailed Agentic AI flowchart is shown in Figure A.9 (Appendix A). We elaborate more about the Agentic AI features in Section 3.6.4.

3.3 Circuit Design Diagram

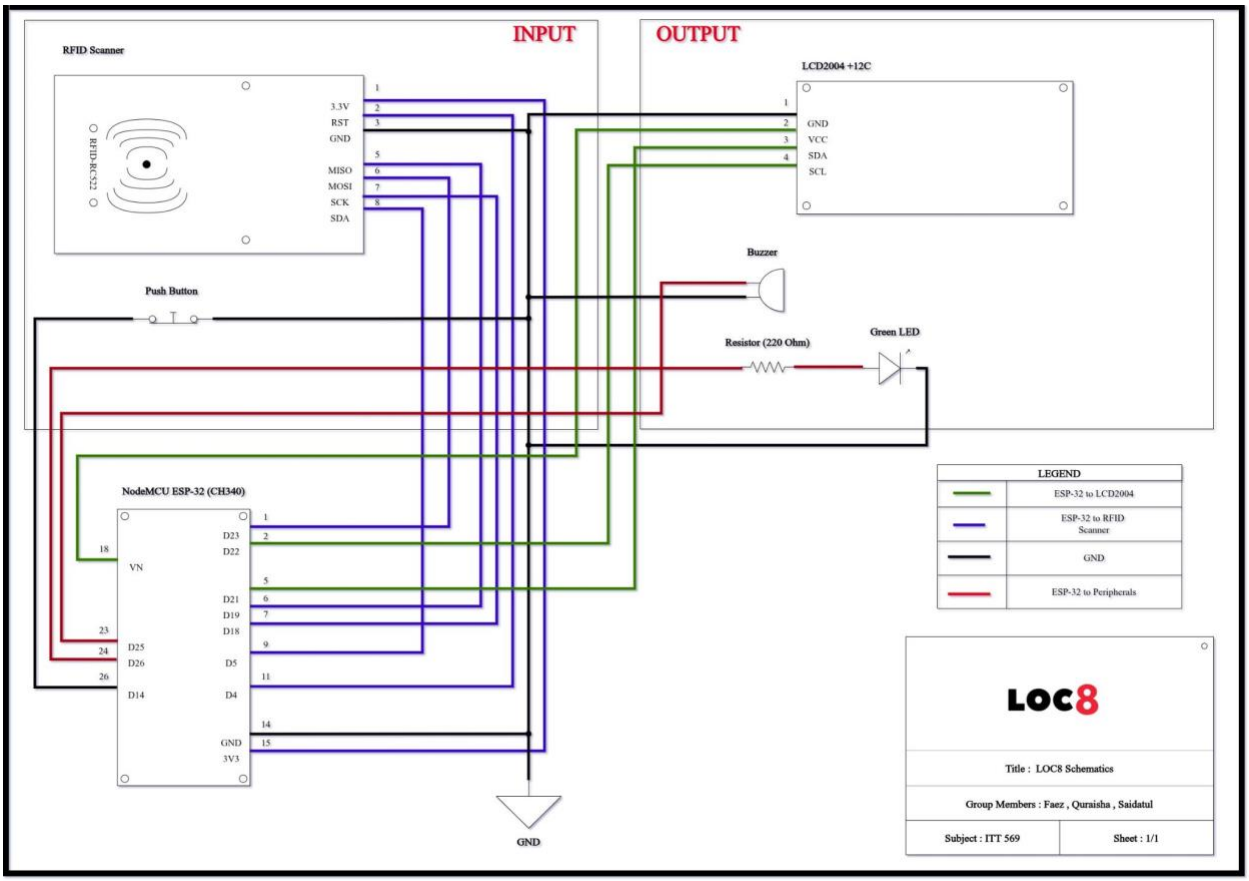


Figure 3.12 : Schematics

Figure 3.12 illustrates the schematic circuit design for the LOC8 parcel management system. The system integrates both input and output components with the NodeMCU ESP-32 (CH340) microcontroller serving as the primary processing unit. The hardware integration is categorized into two main sections:

3.4.1 Input Peripherals

The RFID-RC522 module is utilized for parcel tag detection via the SPI communication protocol while the push button functions as a manual trigger for the register and verify mode.

3.4.2 Output Peripherals

The LCD 2004 (with I2C module) is employed for real-time status display. Additionally, the buzzer and green LED serve as audio and visual indicators to notify users.

To ensure circuit stability, a Common Ground (GND) strategy is applied where all negative pins of the components are connected to a single reference ground point on the ESP-32. A 220 Ω resistor is connected in series with the green LED to function as a current-limiting resistor protecting the component from overcurrent. All pin connections between the ESP-32 and the peripherals are established according to the specific GPIO mapping as illustrated in the figure above.

3.4 Hardware Assembly

Component	Pin (Component)	GPIO Pin (ESP32)	Communication/Note
RFID-RC522	SDA (SS)	GPIO 5	SPI
	SCK	GPIO 18	SPI
	MOSI	GPIO 23	SPI
	MISO	GPIO 19	SPI
	RST	GPIO 4	Reset
	GND	GND	Common Ground
	3.3V	3V3	Power Supply

LCD 2004	SDA	GPIO 21	I2C Data
	SCL	GPIO 22	I2C Clock
	VCC	VIN (5V)	Power Supply
	GND	GND	Common Ground
Buzzer	Positive (+)	GPIO 25	Active Buzzer
	Negative (-)	GND	Common Ground
Green LED	Anode (+)	GPIO 26	via 220 Ω Resistor
	Cathode (-)	GND	Common Ground
Push Button	Terminal 1	GPIO 14	Digital Input
	Cathode (-)	GND	Common Ground

Table 3.13 : Pin Mapping

The initial assembly was conducted using a breadboard environment to facilitate rapid prototyping and logic verification. This stage allowed the team to validate the pin assignments between the ESP32 and peripherals like the MFRC522 and LCD 2004 without making permanent changes as shown in Figure 3.14 and Figure 3.15.

Figure 3.14 : Hardware Setup

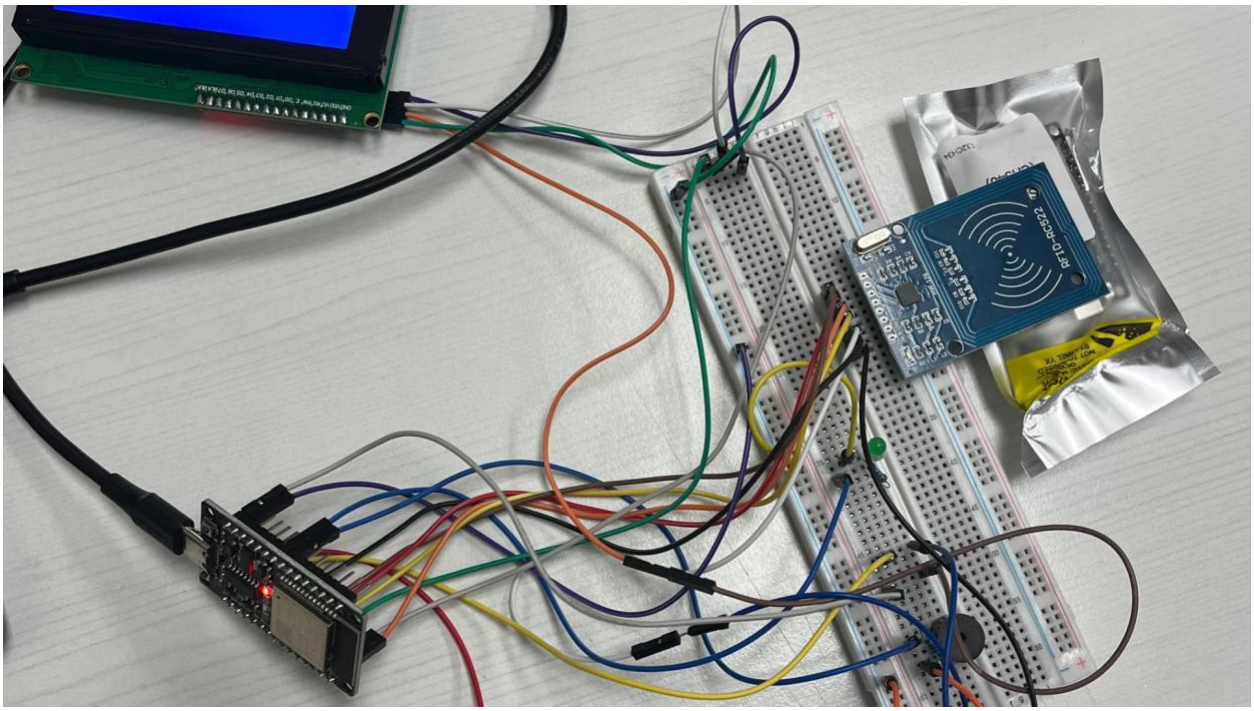


Figure 3.15 : Prototype Implementation of the LOC8 system
on a Breadboard Prior to Final Soldering

To ensure long-term reliability and signal stability, we consulted an electronic shop for professional soldering services as shown in Figure 3.16.
Address : Prosine Electrics & Electronics, 76, Jalan TU 42, Taman Tasik Utama, 75350 Ayer Keroh, Melaka, Malaysia.

Figure 3.16 : Electronic Shop for Hardware Soldering

This transition from a breadboard prototype to a permanent soldered circuit was essential to eliminate intermittent connectivity issues and to ensure the hardware remains robust during high-volume parcel scanning at Gerai Tanjung as shown in Figure 3.17.

Figure 3.17 : Post Hardware Soldering Process

By securing the components onto a dedicated perfboard, the system became resilient to physical handling, ensuring consistent performance for long-term deployment at the parcel hub.

Following the hardware consolidation, the device enclosure was fabricated using 3D printing technology at a shophouse to achieve a professional aesthetic and ergonomic finish as shown in Figure 3.18-3.22.

Address : 3D Print Melaka, No. 8, Jalan Impiana Nilam 2, Taman Impiana Kesang, 77000 Jasin, Melaka

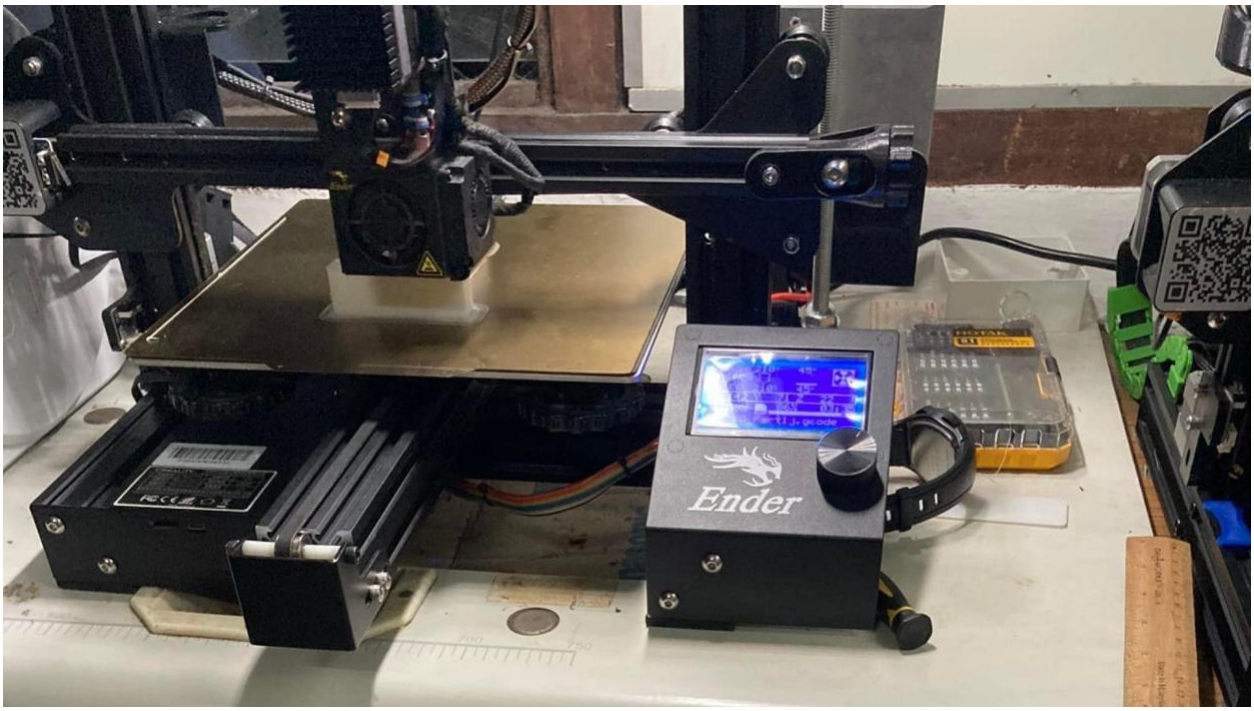


Figure 3.18 : Shophouse for 3D Printing

Figure 3.19 : Fully assembled 3D CAD Model

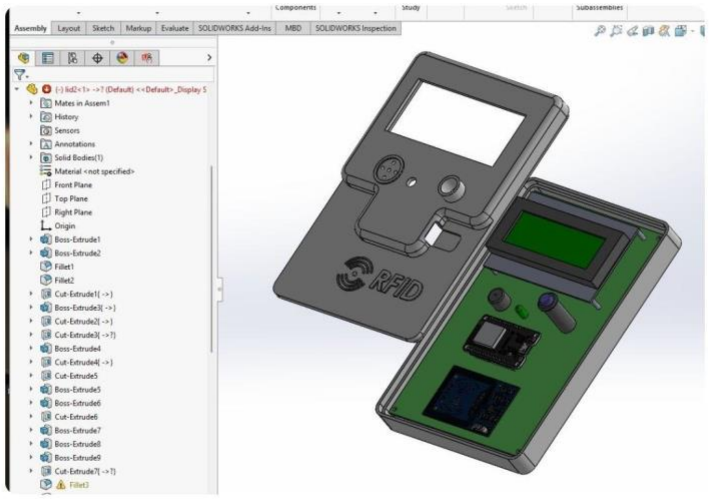


Figure 3.20 : Open Assembly View CAD Model

Figure 3.21 : Transparent Assembly View CAD Model

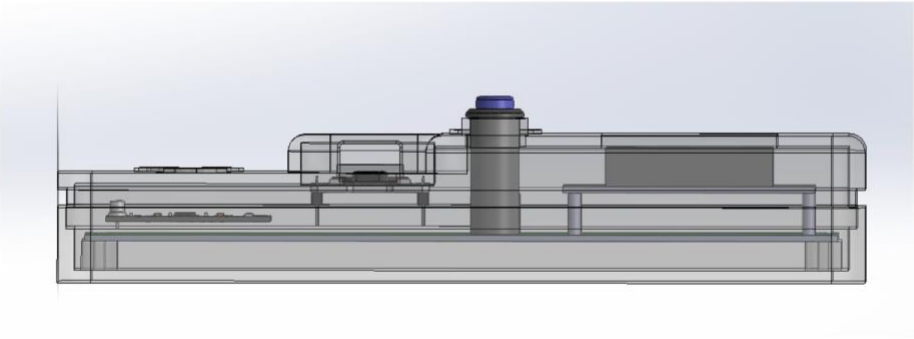


Figure 3.22 : Sectional Side View CAD Model

The structural form of the handheld housing drew creative inspiration from the classic Game Boy console design as shown in Figure 3.23.

Figure 3.23 : Game Boy Console

The casing color scheme featured a vibrant red for the top cover and a sleek black for the bottom base. This specific color selection was intended to align seamlessly with the official LOC8 logo and establish a cohesive and recognizable corporate appearance for the final product as shown in Figure 3.24 & Figure 3.25.

The primary material used to print these components is Polylactic Acid Plus (PLA+).

The selection of PLA+ over conventional PLA is based on its enhanced mechanical properties, which include higher impact resistance, reduced brittleness, and stronger layer adhesion. These characteristics ensure that the fabricated hardware casing is more durable and capable of protecting the internal electronic components (such as the ESP32 and RFID sensors) from external physical stress during system operation.

Figure 3.24 :Front View of Final Product

Figure 3.25 : Inside View of Final Product

3.5 Software Setup

Link to download Arduino IDE: https://www.arduino.cc/en/software/

For Windows just run the installer .exe file.

Since Faez is using Linux (Ubuntu) as his main OS in his computer he has to do a workaround to make the Arduino IDE works. Being on Linux requires handling permissions for the USB port so that the ESP32 can be "seen" by the system. He used this video as a guidance step by step tutorial to setup Arduino IDE.

Video Link : https://youtu.be/1WtWqiaFejg?si=jnvOuMK486d_k_kM Thanks stranger !

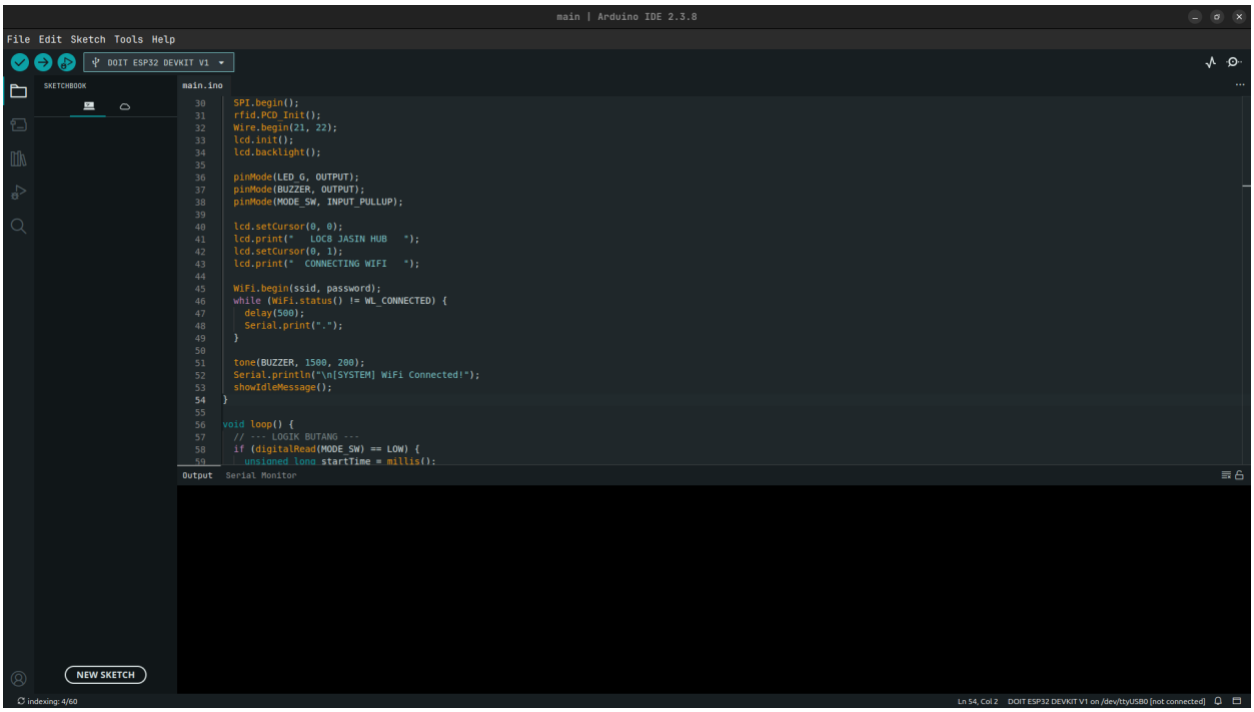


Figure 3.26 : Arduino IDE on Linux (Ubuntu)

3.6 System Implementation

This section describes the deployment of the LOC8 system, detailing the technical bridge between the hardware logic and the cloud-based backend. **All full source codes are within Appendix B.**

3.6.1 Backend API Integration for ESP32

This section delineates the communication protocol between the ESP32 and the Google Sheets cloud database, which serves as the core processing engine for the LOC8 system.

```
if (WiFi.status() == WL_CONNECTED) {
  HTTPClient http;
  WiFiClientSecure client;
  client.setInsecure();

  String url = String(scriptURL) + "?uid=" + uid + "&mode=" + action
+ "&size=" + size;
  Serial.print("[HTTP] DISPATCHING GET REQUEST: ");
  Serial.println(url);

  lcd.clear();
  lcd.setCursor(0, 1);
  lcd.print("  SYNCING CLOUD...  ");

  http.begin(client, url.c_str());
  http.setTimeout(30000); // 30-Second Execution Timeout Guard
  http.setFollowRedirects(HTTPC_FORCE_FOLLOW_REDIRECTS);
  int httpCode = http.GET();
```

Code Snippet 3.27 : HTTP Request Logic to Google Sheet

3.6.2 Verification Logic for ESP32

This section details the decision-making logic employed by the system to determine parcel availability during the retrieval process.

```
DynamicJsonDocument doc(2048);
DeserializationError error = deserializeJson(doc, payload);

if (!error) {
  String status = doc["status"].as<String>();
  status.toUpperCase();

  if (status == "SUCCESS" || status == "VERIFIED" || status ==
"REGISTERED") {
    digitalWrite(LED_G, HIGH);
    tone(BUZZER, 2000, 200);
    // [Hardware Interface Update Subroutine Goes Here]
    delay(4000);
    digitalWrite(LED_G, LOW);
  } else {
    // Failure Warning Sequence: 4 Rapid LED Flashes and Low Beeps
    for (int i = 0; i < 4; i++) {
      digitalWrite(LED_G, HIGH);
      tone(BUZZER, 300, 250);
      delay(250);
      digitalWrite(LED_G, LOW);
      delay(250);
    }
  }
}
```

Code Snippet 3.28 : Verification and Feedback Logic

3.6.3 Data Processing and API Logic for Google Apps Script

1. **Inbound Allotment Mode (mode === "inbound")**

The engine executes a background structural utility (deleteBlankRows()) to purge malformed data. It reads state flags dynamically from PropertiesService (LAST_RACK and LAST_SHELF) to enforce load-balancing inventory optimization across Racks (PIDs 1000–4999) or Shelves (PIDs 5000–8999), executing anti-collision verification via a relational lookup map (activeIDsSet).

2. **Outbound Bulk Extraction Mode (mode === "outbound")**

The backend maps the client uid parameter against the StudentDB repository to resolve student identity arrays. It identifies all active items designated as "IN" tied to that unique profile, executes an atomic single-transaction update changing row properties to "COLLECTED", flushes the spreadsheet data cache container, and dispatches an automatic messaging job.

3. **Card Mapping Mode (mode === "register_card")**

The backend scans the student ledger for empty identifier slots and dynamically binds the unlinked physical card hex to the student data line without overwriting pre-existing structural attributes.

3.6.4 Autonomous Notification Framework (Agentic AI) for Google Apps Script

The system incorporates an event-driven "Agentic AI" framework that functions without human intervention. This is achieved through several triggers.

1. **Instant Event-Driven Trigger (onEdit)**

It serves as a responsive agent that automates student notifications by monitoring real-time updates in the *ParcelDB* spreadsheet. When staff update a parcel's status, the script immediately cross-references the student's CARD UID with the *StudentDB* to retrieve the correct Telegram Chat ID and dispatch an alert. This mechanism eliminates manual messaging and human error, ensuring students are notified instantly upon parcel registration.

2. **Scheduled Autonomous Audit (checkLateParcels)**

The Scheduled Autonomous Audit (checkLateParcels) acts as a background monitoring agent that performs periodic database audits to manage storage efficiency. Using a JavaScript-based aging logic, the system calculates the duration of stay for each parcel; if a parcel remains unclaimed for three days or more, a reminder is automatically dispatched to the student. This daily audit, powered by a time-driven trigger, ensures high parcel turnover and prevents physical congestion within the storage racks without requiring manual supervision.

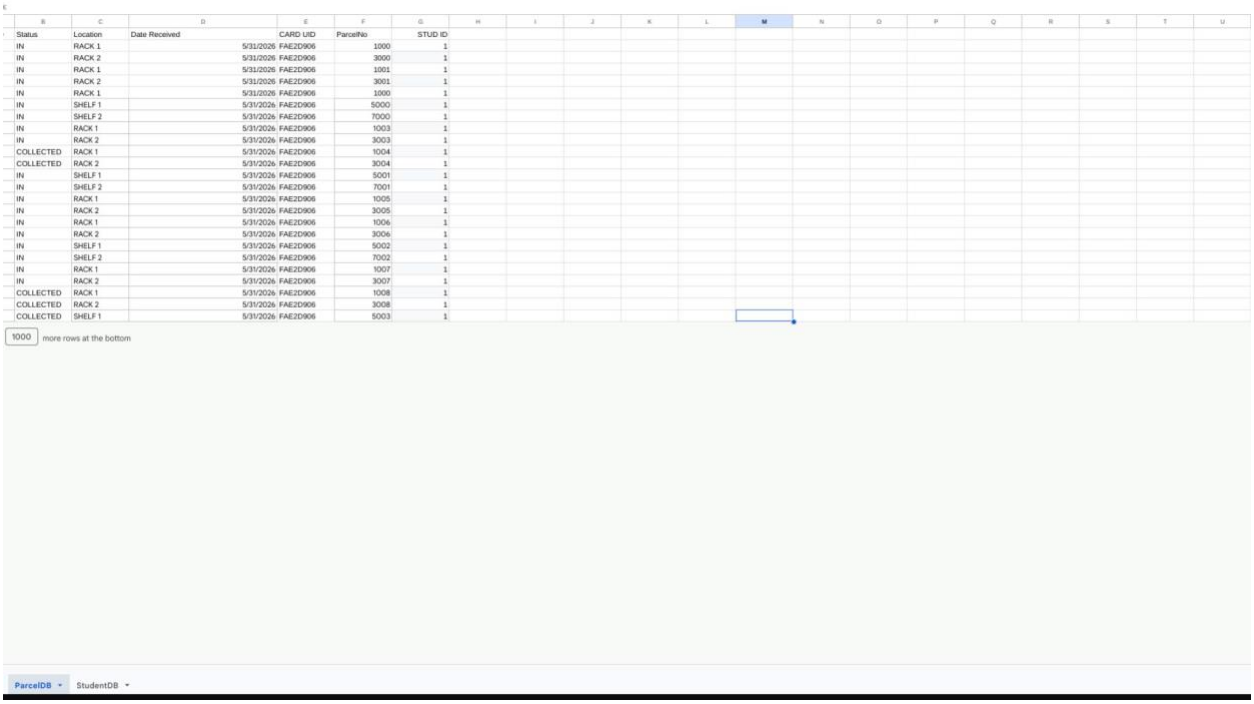
3. **Data Integrity & Telegram Integration**

The Data Integrity & Telegram Integration component, powered by the sendTelegramAlert utility, ensures robust and transparent communication. It incorporates automatic data filtering to validate against undefined or null values, preventing API execution failures and maintaining system stability. Furthermore, it provides administrative oversight by simultaneously carbon-copying every student notification to the system administrator, creating a transparent audit trail for monitoring purposes.

3.7 Google Sheet (Cloud Database)

Google Sheets was selected as the cloud-based database for this project because it is free, easy to use, and supports real-time data synchronization. It enables parcel information to be stored, updated, and accessed online without requiring a dedicated database server. In addition, Google Sheets can be integrated with ESP32 through Google Apps Script, allowing parcel records to be updated automatically after an RFID card is scanned. The system uses two Google Sheets databases.

The first sheet is the **Parcel Database** as shown in Figure 3.29, which stores parcel-related information, including **Parcel UID, Status, Location, Date Received, Card UID,** and **Parcel Number**. This information is used to record and monitor the status of each parcel.



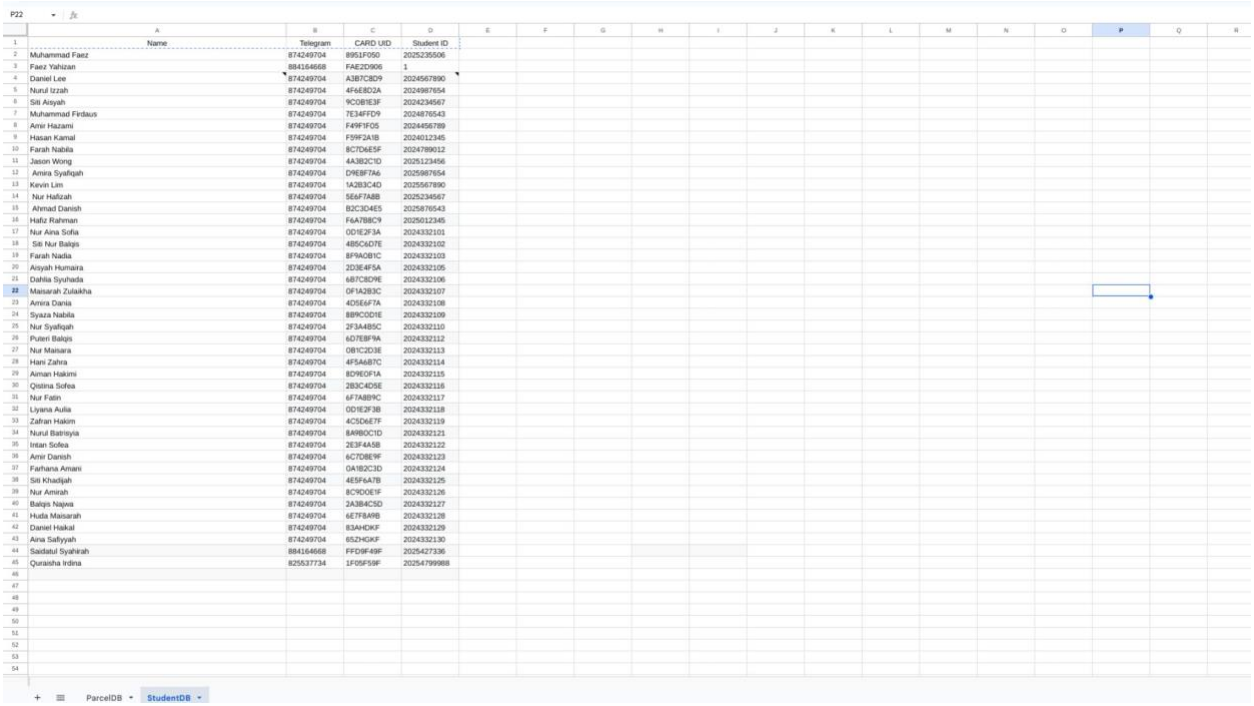
Status	Location	Date Received	CARD UID	ParcelNo	STUD ID
IN	RACK 1	5/31/2026	FAE2D906	1000	1
IN	RACK 2	5/31/2026	FAE2D906	3000	1
IN	RACK 1	5/31/2026	FAE2D906	1001	1
IN	RACK 2	5/31/2026	FAE2D906	3001	1
IN	RACK 1	5/31/2026	FAE2D906	1000	1
IN	SHELF 1	5/31/2026	FAE2D906	5000	1
IN	SHELF 2	5/31/2026	FAE2D906	7000	1
IN	RACK 1	5/31/2026	FAE2D906	1003	1
IN	RACK 2	5/31/2026	FAE2D906	3003	1
COLLECTED	RACK 1	5/31/2026	FAE2D906	1004	1
COLLECTED	RACK 2	5/31/2026	FAE2D906	3004	1
IN	SHELF 1	5/31/2026	FAE2D906	5001	1
IN	SHELF 2	5/31/2026	FAE2D906	7001	1
IN	RACK 1	5/31/2026	FAE2D906	1005	1
IN	RACK 2	5/31/2026	FAE2D906	3005	1
IN	RACK 1	5/31/2026	FAE2D906	1006	1
IN	RACK 2	5/31/2026	FAE2D906	3006	1
IN	SHELF 1	5/31/2026	FAE2D906	5002	1
IN	SHELF 2	5/31/2026	FAE2D906	7002	1
IN	RACK 1	5/31/2026	FAE2D906	1007	1
IN	RACK 2	5/31/2026	FAE2D906	3007	1
COLLECTED	RACK 1	5/31/2026	FAE2D906	1008	1
COLLECTED	RACK 2	5/31/2026	FAE2D906	3008	1
COLLECTED	SHELF 1	5/31/2026	FAE2D906	5003	1

1000 more rows at the bottom

ParcelDB StudentDB

Figure 3.29 : Parcel Database (ParcelDB)

The second sheet is the **Student Database** as shown in Figure 3.30, which contains student information, including **Student ID, Name, Telegram,** and **Card UID**. The Card UID acts as a unique identifier that links each RFID card to a registered student.



Name	Telegram	CARD UID	Student ID
Muhammad Faez	874249704	8951F050	2025235506
Faez Yahizan	884164668	FAE2D906	1
Daniel Lee	874249704	A3B7C8D9	2024567890
Nurul Izzah	874249704	4F6E8D2A	2024987654
Siti Aisyah	874249704	9C0B1E3F	2024234567
Muhammad Firdaus	874249704	7E34FFD9	2024876543
Amir Hazami	874249704	F49F1F05	2024456789
Hasan Kamal	874249704	F59F2A1B	2024012345
Farah Nabila	874249704	8C7D6E5F	2024789012
Jason Wong	874249704	4A3B2C1D	2025123456
Amira Syafiqah	874249704	D9EBF7A6	2025987654
Kevin Lim	874249704	1A2B3C4D	2025567890
Nur Hafizah	874249704	5E6F7A8B	2025234567
Ahmad Danish	874249704	B2C3D4E5	2025876543
Hafiz Rahman	874249704	F6A7B8C9	2025012345
Nur Aina Sofia	874249704	0D1E2F3A	2024332101
Siti Nur Balqis	874249704	4B5C6D7E	2024332102
Farah Nadia	874249704	BF9A0B1C	2024332103
Aisyah Humaira	874249704	2D3E4F5A	2024332105
Dahlia Syuhada	874249704	6B7C8D9E	2024332106
Maisarah Zulaikha	874249704	0F1A2B3C	2024332107
Amira Dania	874249704	4D5E6F7A	2024332108
Syaza Nabila	874249704	8B9C0D1E	2024332109
Nur Syafiqah	874249704	2F3A4B5C	2024332110
Puteri Balqis	874249704	6D7E8F9A	2024332112
Nur Maisara	874249704	0B1C2D3E	2024332113
Hani Zahra	874249704	4F5A6B7C	2024332114
Aiman Hakimi	874249704	8D9E0F1A	2024332115
Qistina Sofea	874249704	2B3C4D5E	2024332116
Nur Fatin	874249704	6F7A8B9C	2024332117
Liyana Aulia	874249704	0D1E2F3B	2024332118
Zafran Hakim	874249704	4C5D6E7F	2024332119
Nurul Batrisyia	874249704	8A9B0C1D	2024332121
Intan Sofea	874249704	2E3F4A5B	2024332122
Amir Danish	874249704	6C7D8E9F	2024332123
Farhana Amani	874249704	0A1B2C3D	2024332124
Siti Khadijah	874249704	4E5F6A7B	2024332125
Nur Amirah	874249704	8C9D0E1F	2024332126
Balqis Najwa	874249704	2A3B4C5D	2024332127
Huda Maisarah	874249704	6E7F8A9B	2024332128
Daniel Haikal	874249704	83AHDKF	2024332129
Aina Safiyyah	874249704	65ZHGKF	2024332130
Saidatul Syahirah	884164668	FFD9F49F	2025427336
Quraisha Irdina	825537734	1F05F59F	20254799988

ParcelDB StudentDB

Figure 3.30 : Student Database (StudentDB)

By connecting these two databases, the system can automatically match the scanned RFID Card UID with the corresponding student record and retrieve the associated parcel information. The stored data are then synchronized with the web-based dashboard, allowing administrators to monitor parcel records and student information in real time. This approach reduces manual data entry, minimizes human errors, and improves the efficiency of parcel management.

3.8 Website

Website : loc8tanjung.site

3.8.1 Main Page

The website was developed as a web-based platform to provide users with access to the parcel management system. The website consists of several navigation sections, including Performance, Features, Group Members, FAQ, and Parcel Monitoring System as shown in Figure 3.31-3.33 . These sections provide information about the project while enabling students to access parcel-related services through a single platform.

Figure 3.31 : Website (Main Page)

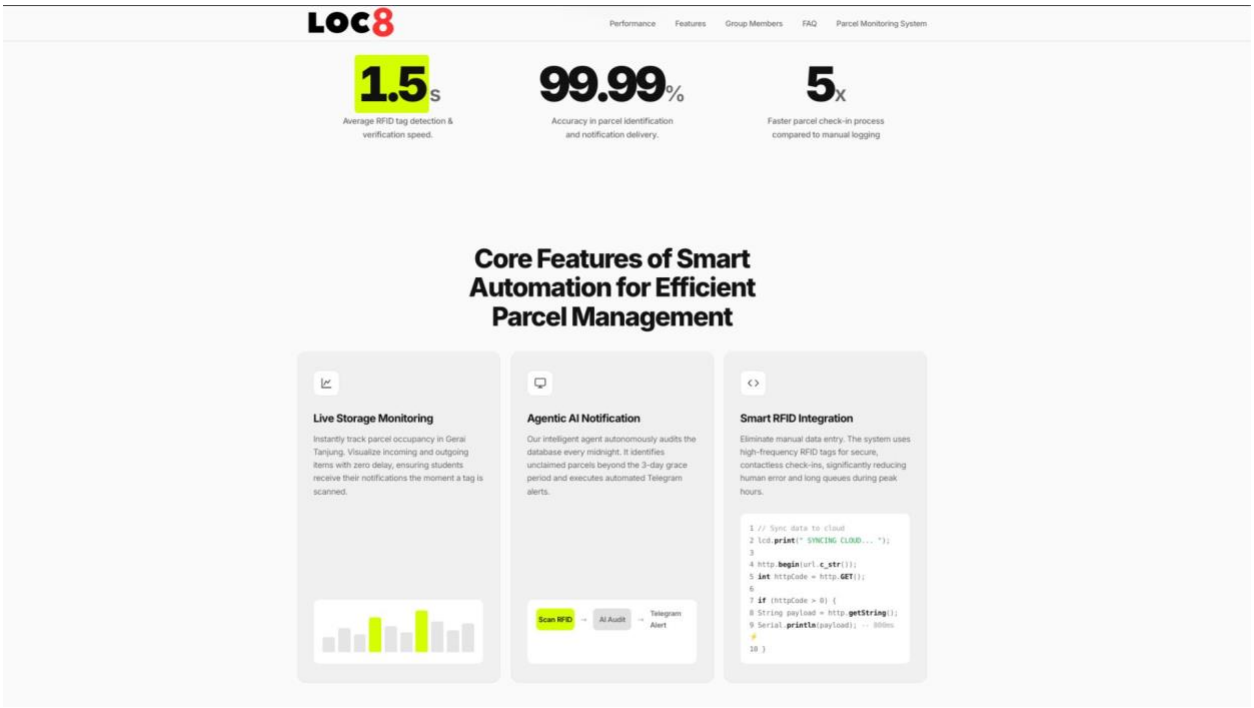


Figure 3.32 : Performance & Core Features (Main Page)

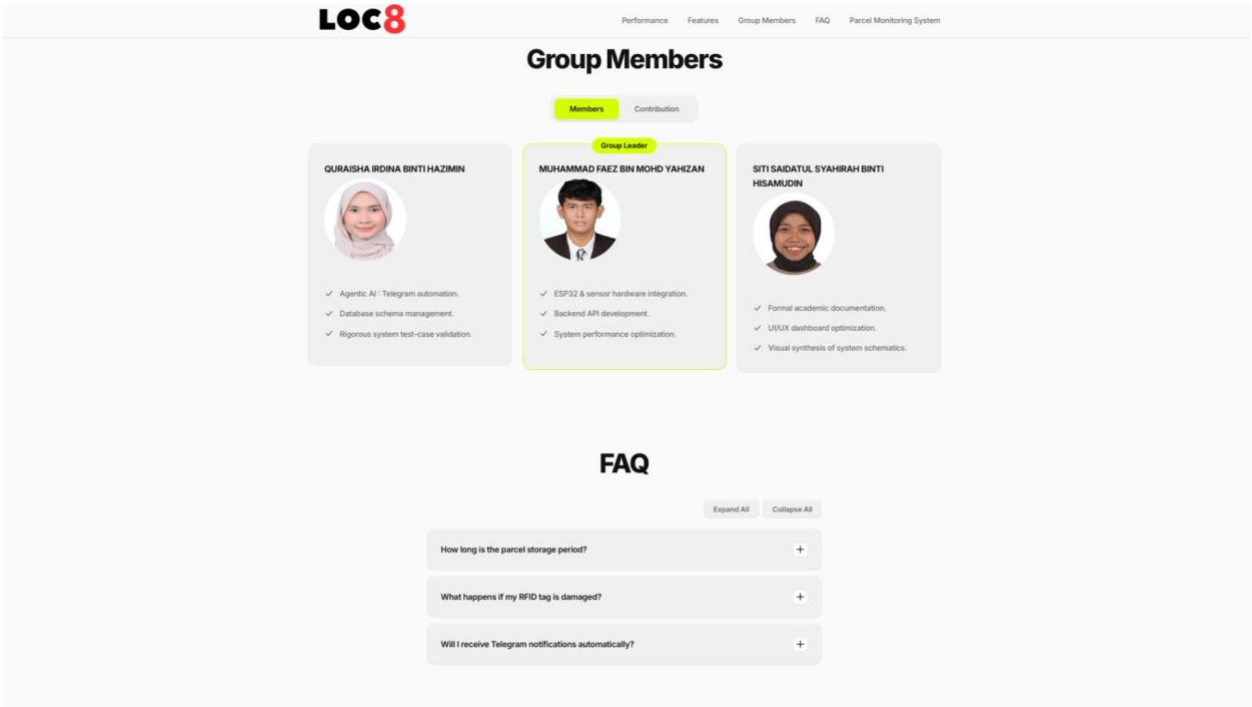


Figure 3.33: Group Members & FAQ (Main Page)

3.8.2 Parcel Monitoring System (Dashboard)

The **Parcel Monitoring System** section is the main functional component of the website as shown in Figure 3.34. It retrieves parcel and student data from Google Sheets through an Application Programming Interface (API) and displays the latest information in real time. Students can use this feature to check the status, collection location, and date received of their parcels without visiting the parcel collection counter.

Additionally, the system provides a search and filtering function that allows students to enter their **Student ID** to retrieve only the parcel records associated with their account. This feature enables students to quickly determine whether their parcels have arrived and are ready for collection, while ensuring that the displayed information is relevant to the individual user.

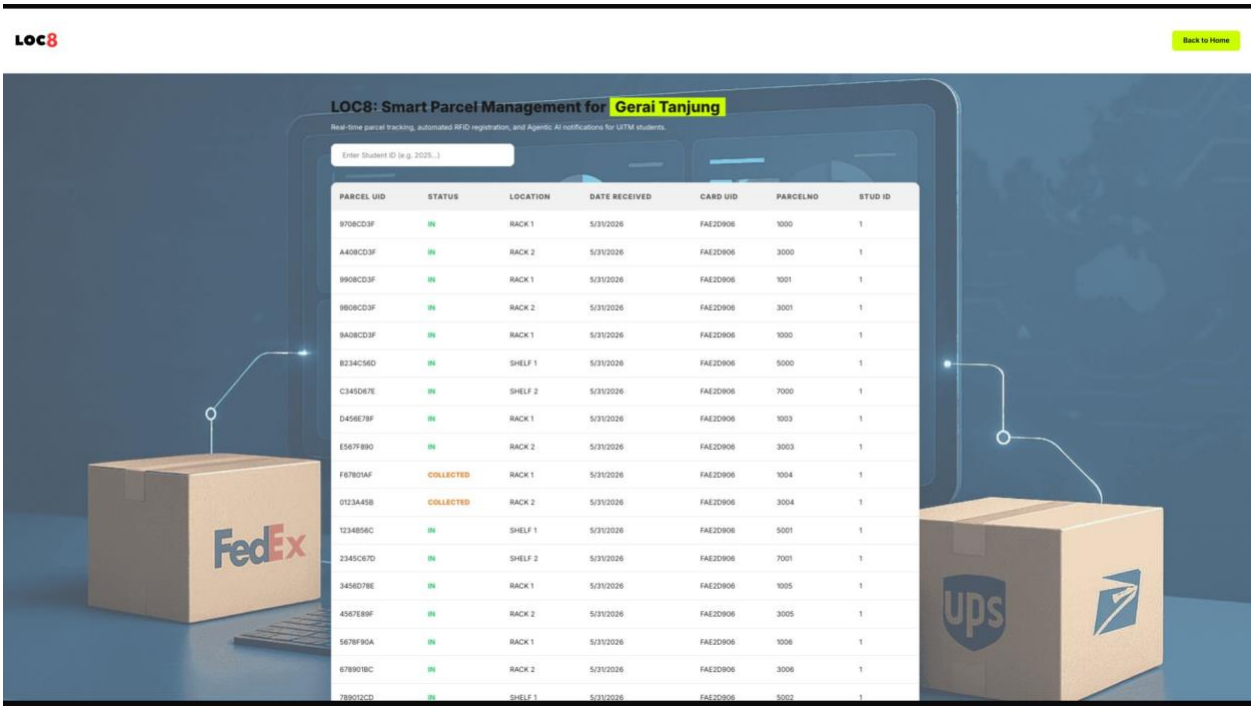


Figure 3.34 : Parcel Monitoring System (Dashboard)

3.9 Project Planning

The development of the LOC8 system follows a structured 8-week timeline, beginning from the project title approval in Week 4 until the final product testing in Week 14. Table 3.35 illustrates the detailed Gantt Chart highlighting the key tasks.

Phase	Task	Start Week	End Week	Start Date	End Date
DESIGN	Pre-Project & Title Proposal	W1	W3	30 Mac	19 Apr
	Project Title Approval & Kick-off	W4	W4	20 Apr	26 Apr
	Concept & Requirement Analysis	W5	W5	27 Apr	3 Mei
	Market Research (User Survey)	W6	W6	4 Mei	10 Mei
	Circuit Design & Schematics (ESP32)	W7	W7	11 Mei	17 Mei
PROTOTYPE	Hardware Selection & Pin Mapping	W8	W8	18 Mei	24 Mei
	MID-SEMESTER & FESTIVE BREAK	Break		25 Mei	2 Jun

	Backend & Cloud Integration (Google Sheets)	W9	W9	3 Jun	9 Jun
	Build Physical Prototype & Soldering	W10	W11	10 Jun	23 Jun
	System Integration & Debugging	W12	W12	24 Jun	30 Jun
PRODUCTION	Final Testing & Video Demonstration	W13	W13	1 Jul	7 Jul
	Final Submission & Ready for Exhibition	W14	W14	8 Jul	14 Jul

Table 3.35 : Gantt Chart for Project Planning

CHAPTER 4:TESTING AND FINDING

This chapter presents the verification phases, test execution matrices, and empirical findings for the LOC8 Smart Parcel System. Evaluation was structured into three distinct test groups: Hardware Subsystem Validation (HW), End-to-End Cloud API Integration (INT), and Automated Telegram Notification Triggering (NOT). These systematically mapped test cases ensure complete data integrity between physical peripheral actions and cloud repository updates.

4.1 Test Execution

4.1.1 Subsystem Testing (Hardware Validation Matrix)

This section evaluates the functionality of individual hardware peripheral devices connected to the ESP32 microcontroller unit (MCU). Testing confirms that GPIO pin mapping, signal logic, and peripheral initialization match the structural design criteria.

Table 4.1 categorizes the verification criteria executed at the hardware subsystem level.

Test ID	Subsystem / Component	Test Procedure	Expected Outcome	Actual Result	Status
HW-01	MFRC522 RFID Reader	Scan an RFID tag or student card near the RC522 module (SS_PIN: 5, RST_PIN: 4).	The module reads the raw hexadecimal UID bytes, cleans the formatting, converts any alphabetic characters to uppercase, and prints the clean string to the Serial Monitor.	The clean UID string is successfully captured and displayed without hyphens.	PASSED

Table 4.1 : Subsystem Hardware Validation

Test ID	Subsystem / Component	Test Procedure	Expected Outcome	Actual Result	Status
HW-02	I2C LCD Display	Initialize the LCD via I2C (SDA: 21, SCL: 22) at address 0x27 and display the system startup message.	The LCD backlight initializes, and text strings appear clearly across the rows (“ LOC8 JASIN HUB “).	Text is rendered correctly with accurate positioning across all 4 rows.	PASSED

Table 4.1 : (continued)

Figure 4.1.2 HW-02

Test ID	Subsystem / Component	Test Procedure	Expected Outcome	Actual Result	Status
HW-03	Mode Selector Switch	Execute a short press on the physical tactile pull-up button.	The system advances the global currentMode variable triggers a 1000Hz short beep, and updates the LCD idle screen.	currentMode increments sequentially, and state transition changes reflect instantly.	PASSED

Table 4.1 : (continued)

Test ID	Subsystem / Component	Test Procedure	Expected Outcome	Actual Result	Status
HW-04	Size Toggle Logic	Execute a long press (hold duration $> 1500ms on the mode switch while the system is under Mode 1.	The system flips the isBigSize flag, triggers a 1200Hz confirmation tone, and switches the allocation display between "BESAR (SHELF)" and "RINGAN (RACK)".	The system accurately filters press duration, preventing unintentional core mode skips during storage sizing adjustment.	PASSED

Table 4.1 : (continued)

Figure 4.1.3 HW-04

Test ID	Subsystem / Component	Test Procedure	Expected Outcome	Actual Result	Status
HW-05	Status Indicator & Alert Peripherals	Trigger success and error subroutines via the green LED (LED_G: 26) and Piezo Buzzer (BUZZER: 25).	Success: Solid LED with a high-pitched 2kHz chirp. Failure: 4 rapid structural blinks with 300Hz warning low-tones.	Peripherals respond immediately with distinct visual and acoustic signatures based on system execution.	PASSED

Table 4.1 : (continued)

4.1.2 Integration and Cloud Testing (End-to-End API Validation)

This phase validates the continuous data handshake, structural redirection, and JSON payload deserialization between the ESP32 client and the deployment endpoint of the Google Apps Script Web API via secure HTTPS.

Table 4.2 presents the evaluation protocol implemented to validate cloud database operations.

Test ID	Integration Scenario	API Parameter Payload	Expected Cloud & Hardware Behavior	Verification Status
INT-01	Mode 1: Parcel Inbound Allocation (Light Item)	?uid=[UID]&mode=inbound&size=light	The script queries ParcelDB. If the item is new, it generates an incremental Parcel ID (PID) within the 1000–4999 Rack range. ESP32 parses doc["status"] == "SUCCESS", illuminates LED_G, and the LCD instructs the operator to write the PID number on the box.	PASSED

Table 4.2 : Validate cloud database operations

Test ID	Integration Scenario	API Parameter Payload	Expected Cloud & Hardware Behavior	Verification Status
INT-02	Mode 1: Parcel Inbound Allocation (Large Item)	?uid=[UID]&mode=inbound&size=big	The script dynamically toggles properties LAST_SHELF between 1 and 2, assigns a PID within the 5000–8999 Shelf range, and updates row records. ESP32 decodes the JSON payload and outputs the specific shelf location onto the screen.	PASSED

Table 4.2: continued

Test ID	Integration Scenario	API Parameter Payload	Expected Cloud & Hardware Behavior	Verification Status
INT-03	Mode 1: Duplicate Inbound Prevention	?uid=[UID]&mode=inbound&size=[ANY]	The script catches a pre-existing matching UID with an active "IN" status flag. The cloud returns {"status":"FAILED", "msg":"PARCEL DAH ADA"}. The ESP32 triggers a failure blink sequence.	PASSED

Table 4.2: continued

Figure 4.2.3 INT-03

Test ID	Integration Scenario	API Parameter Payload	Expected Cloud & Hardware Behavior	Verification Status
INT-04	Mode 2: Bulk Outbound Verification & Extraction	?uid=[STUDENT_CARD_UID]&mode=outbound	The script maps the scanned card to StudentDB to discover the Student ID. It batch-updates all matching active package rows from "IN" to "COLLECTED" in a single transaction. The ESP32 extracts the parsed doc["collected_pids"] array and lists the items across the LCD lines.	PASSED

Table 4.2: continued

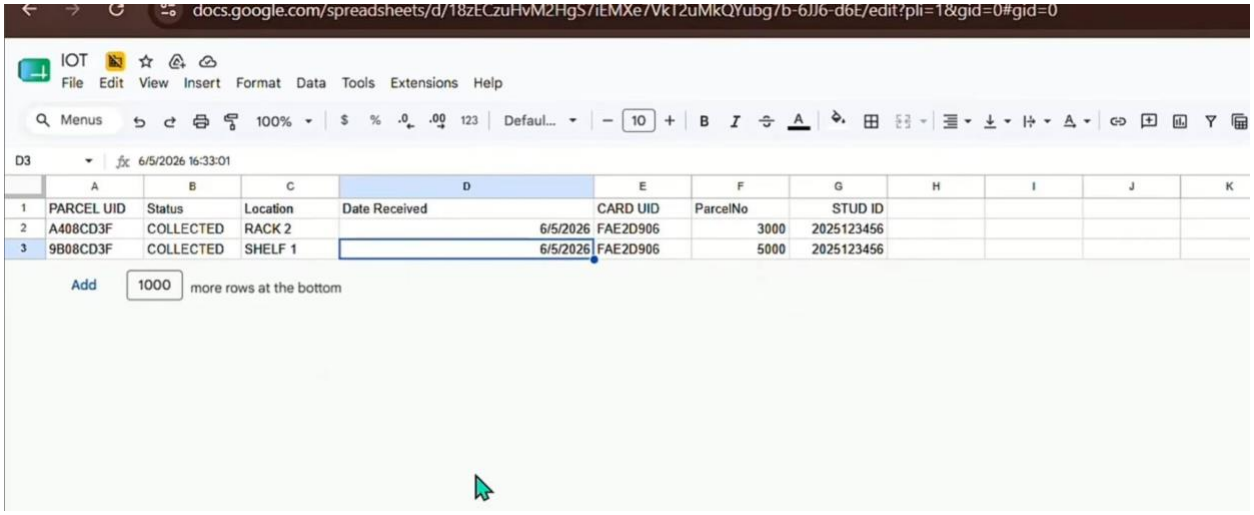


Figure 4.2.4 INT-04

Test ID	Integration Scenario	API Parameter Payload	Expected Cloud & Hardware Behavior	Verification Status
INT-05	Mode 3: Untagged Card Registration	?uid=[NEW_UID]&mode=register_card	The script processes StudentDB for an open, unlinked student row slot and inserts the new UID hex. The database flushes and returns "REGISTERED". The hardware validates the update and outputs "DB STATUS: LINKED".	PASSED

Table 4.2: continued

Figure 4.2.5 INT-05

Test ID	Integration Scenario	API Parameter Payload	Expected Cloud & Hardware Behavior	Verification Status
INT-06	Network Exception & Timeout Handling	Forced disconnect or server lag exceeding setTimeout(30000)	The http.GET() routine drops securely, handles the negative error code, and enters an emergency loop on the ESP32, pulsing the LED at a high frequency without locking the main thread.	PASSED

Table 4.2: continued

4.1.3 Automated Alerting and Notification Testing

This section validates the integration of the Google Apps Script background logic with the external Telegram Bot API. Testing ensures that event-driven messaging triggers and time-driven automated cron-jobs successfully parse sheet parameters into user-end alerts without introducing operational blocks.

Table 4.3 documents the functional testing criteria for the communication notification subsystems.

Test ID	Notification Scenario	Automation / Trigger Event	Expected Telegram Output & Log	Actual Result	Status
NOT-01	Consolidated Bulk Checkout Alert	Invoked automatically during Mode 2 outbound execution via triggerBulkAlert().	The bot pulls studentChatID directly from memory, bypasses repetitive cell-reading loops, and sends one unified message listing all checked-out PIDs to prevent API rate-limits or timeout lags.	A single message containing multiple PIDs ([#1001,1002,3001]) is delivered instantly to the student and the admin log.	PASSED

Table 4.3 :Automated Telegram Notification Validation

Figure 4.3.1 NOT-01

Test ID	**Notification Scenario**	**Automation / Trigger Event**	**Expected Telegram Output & Log**	**Actual Result**	**Status**
NOT-02	Overdue Parcel Scanner (Cron-Job)	Time-driven trigger invoking checkLateParcels() daily (calculated via mathematical epoch offsets.	The system flags active packages with a status of "IN" that have stayed over 3 days, extracts the linked student profiles, and pushes an automated warning alert.	The bot accurately matches dates and broadcasts the specified overdue message: "BELUM DIAMBIL MELEBIHI 3 HARI...".	PASSED

Table 4.3: continued

Figure 4.3.2 NOT-02

Test ID	**Notification Scenario**	**Automation / Trigger Event**	**Expected Telegram Output & Log**	**Actual Result**	**Status**
NOT-03	Manual Vendor Update Alert	Triggered via autoTelegramOnEdit(e) whenever a vendor manually enters a Student ID in Column 7 of ParcelDB.	The spreadsheet interceptor detects the live change, forces a sheet flush, updates the package status row to "IN", and triggers an immediate single checkout alert (triggerSingleAlert).	The system detects the cell edit dynamically and pushes a message stating: "PARCEL TELAH SEDIA UNTUK DIAMBIL".	PASSED

Table 4.3: continued

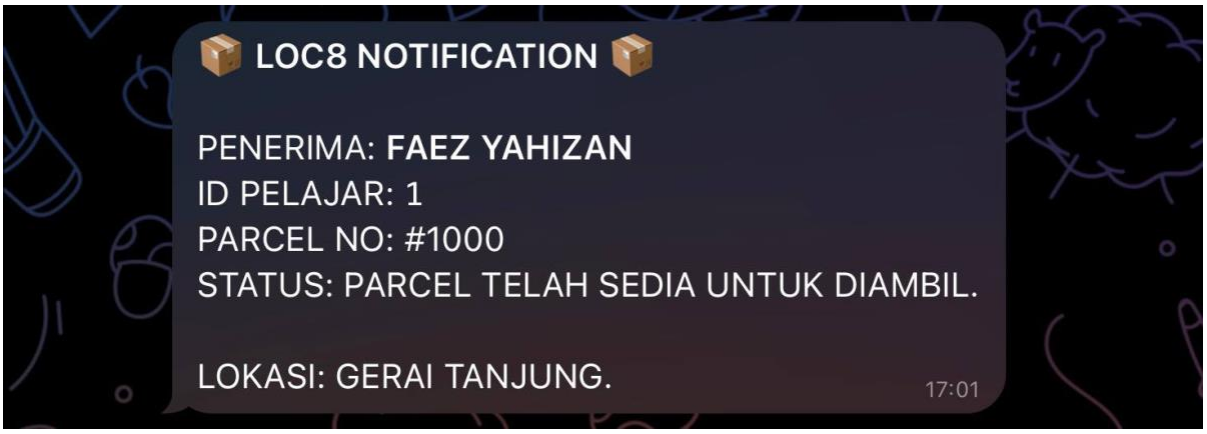


Figure 4.3.3 NOT-03

Test ID	Notification Scenario	Automation / Trigger Event	Expected Telegram Output & Log	Actual Result	Status
NOT-04	Administrative Log Replication	Triggered inside sendTelegramAlert() during any successful alert generation.	The system checks if ADMIN_ID is active. If true, it duplicates the exact user notification payload, adds the prefix 📢 *ADMIN COPY*, and dispatches it to the admin console.	The system successfully handles parallel message routing, delivering copies to the admin without disrupting the student's message thread.	PASSED

Table 4.3: continued

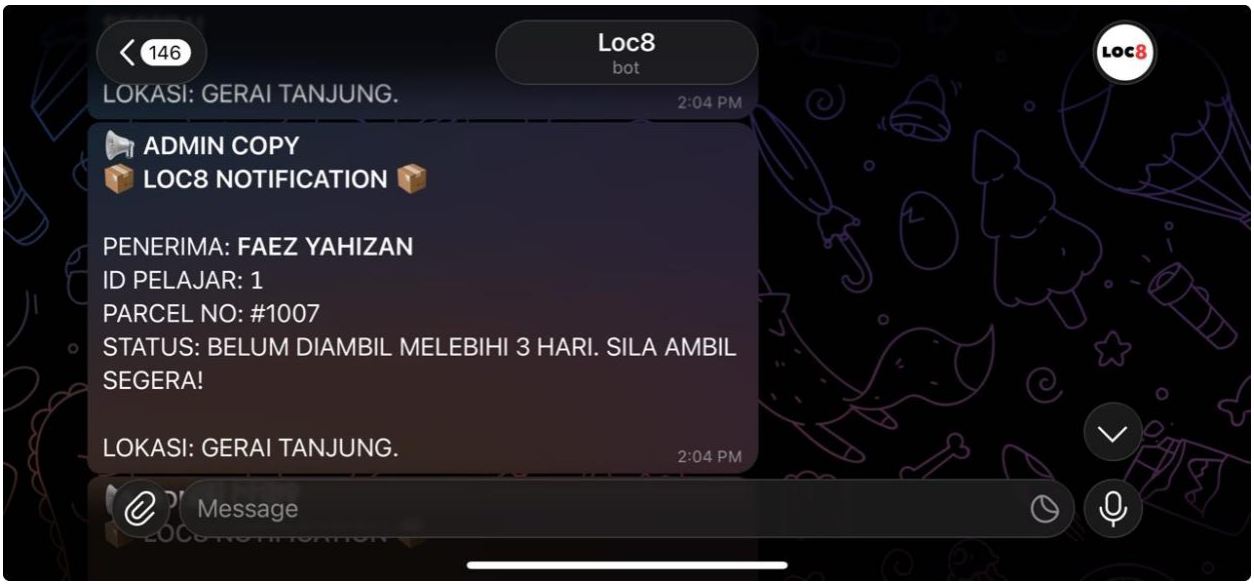


Figure 4.3.4 NOT-04

Test ID	Notification Scenario	Automation / Trigger Event	Expected Telegram Output & Log	Actual Result	Status
NOT-05	Null/Malformed Chat ID Exception	Triggered when chatID is missing, corrupted, or evaluated as "undefined" / < 5 characters.	The validation filter rule `if (!id			if (!id || id == "undefined" || id == "") return;

Table 4.3: continued

4.2 Result Analysis

4.2.1 Experimental Verification Log and Protocol Analysis

1. **Anti-Timeout Consolidated Bulk Alerts (triggerBulkAlert)**

Experimental Log: Testing a multi-parcel outbound checkout proved that instead of sending sequential HTTP GET requests that risk ESP32 connection timeouts, the system intercepts and aggregates all package parameters directly from runtime memory to dispatch a single, consolidated Markdown notification to both the student and admin within 2.5 seconds of the card scan.

2. **Automated Overdue Chrono-Monitor Validation (checkLateParcels)**

Experimental Log: Injected mock records backdated by $\Delta t \geq 3$ days successfully verified that the background script accurately computes storage stay duration using standard epoch offsets via the formula

$$\Delta t = \text{timetoday} - \text{timeparcelin (in days)}$$

automatically isolating the corresponding student metadata to push overdue warning notifications to both the student and administrator channels.

4.2.2 Asynchronous Communication and Network Latency Optimization

1. **Client-Side Timeout Prevention**

During a bulk checkout, the ESP32 executes only a single HTTPS GET request instead of multiple sequential loops. The cloud backend processes all multi-row updates in the runtime memory via triggerBulkAlert(), allowing the hardware client to safely terminate its network session within 2.5 seconds and completely avoiding ESP32 connection timeouts.

2. **Decentralized Background Auditing**

The overdue parcel audit operates entirely as an isolated cloud cron-job via checkLateParcels() without involving the hardware client. The system computes storage duration using standard epoch time deltas ($\Delta t = t_{today} - t_{in}$); if $\Delta t \geq 3$ days, it automatically routes JSON-encoded packets to the Telegram Bot API to alert the student and administrator concurrently, ensuring zero-touch operational automatio

CHAPTER 5: CONCLUSION AND RECOMMENDATIONS

5.1 Conclusion

The development and deployment of the LOC8 Smart Parcel Management System successfully achieved all primary research objectives, providing a comprehensive solution to the operational bottlenecks identified at the Gerai Tanjung, UiTM Jasin. By replacing the error-prone manual logbooks with a cloud-synchronized Google Sheets database ecosystem, the digitization gap in local campus logistics has been effectively bridged.

From a hardware prototyping perspective, rigorous subsystem validation confirmed that proper peripheral routing, precise GPIO pin allocation, and current-limiting circuit safety mechanisms resolved potential pin execution conflicts, resulting in a robust, field-ready IoT handheld client. Furthermore, the integration of an event-driven, autonomous background infrastructure achieved through specialized script handlers proved that real-time notification pipelines can be established with minimal network footprint.

Ultimately, LOC8 successfully demonstrates how combining cost-effective edge-computing devices with serverless automation logic can drastically optimize service delivery speed, prevent human transcription errors, and introduce zero-touch transparency into campus student services.

5.2 Recommendations

For future development, it is recommended that the system:

1. **Transition to Dedicated PCB and Advanced Battery Management Systems**

   Future hardware iterations should move away from prototype perfboards toward a commercially manufactured Printed Circuit Board (PCB). Integrating a dedicated step-up voltage regulator alongside a sophisticated Lithium-Polymer (LiPo) Battery Management System (BMS) with overcharge protection will maximize the handheld unit's operational longevity, ensuring it remains fully autonomous during extended usage phases at the distribution hub.

2. **Migration to a Dedicated Relational Database Management System (RDBMS)**

   While Google Sheets offers high initial agility and accessible API boundaries for prototyping, high-frequency logistics scales benefit from a dedicated database backend such as a Dockerized MySQL or PostgreSQL instance. This migration will bypass spreadsheet cell limit thresholds and eliminate Google API quota restrictions, allowing the system to handle thousands of concurrent tracking lines without latency degradation.

3. **Implementation of UHF RFID and Bulk Antennas**

   Upgrading the current High-Frequency (13.56 MHz) MFRC522 reader to an Ultra-High Frequency (UHF) alternative operating between 860–960 MHz will exponentially increase the spatial read range. This adjustment will allow staff to scan entire batches of arriving parcel baskets from a distance of several meters simultaneously without requiring individual point-blank item placement, radically optimizing inbound turnover.

REFERENCES

High Frequency ISO1443A RFID 13.56MHZ NFC213 144 Bytes Round 25mm Wet Inlay Tag. (2022). Www.alibaba.com. https://www.alibaba.com/product-detail/High-Frequency-ISO1443A-RFID-13-56MHZ_1601563863461.html?spm=a2700.prosearch.normal_offer.d_image.142667afooqEgD&priceId=485e392a48a646db86b7217412780e85

APPENDIX A : ADDITIONAL FIGURES

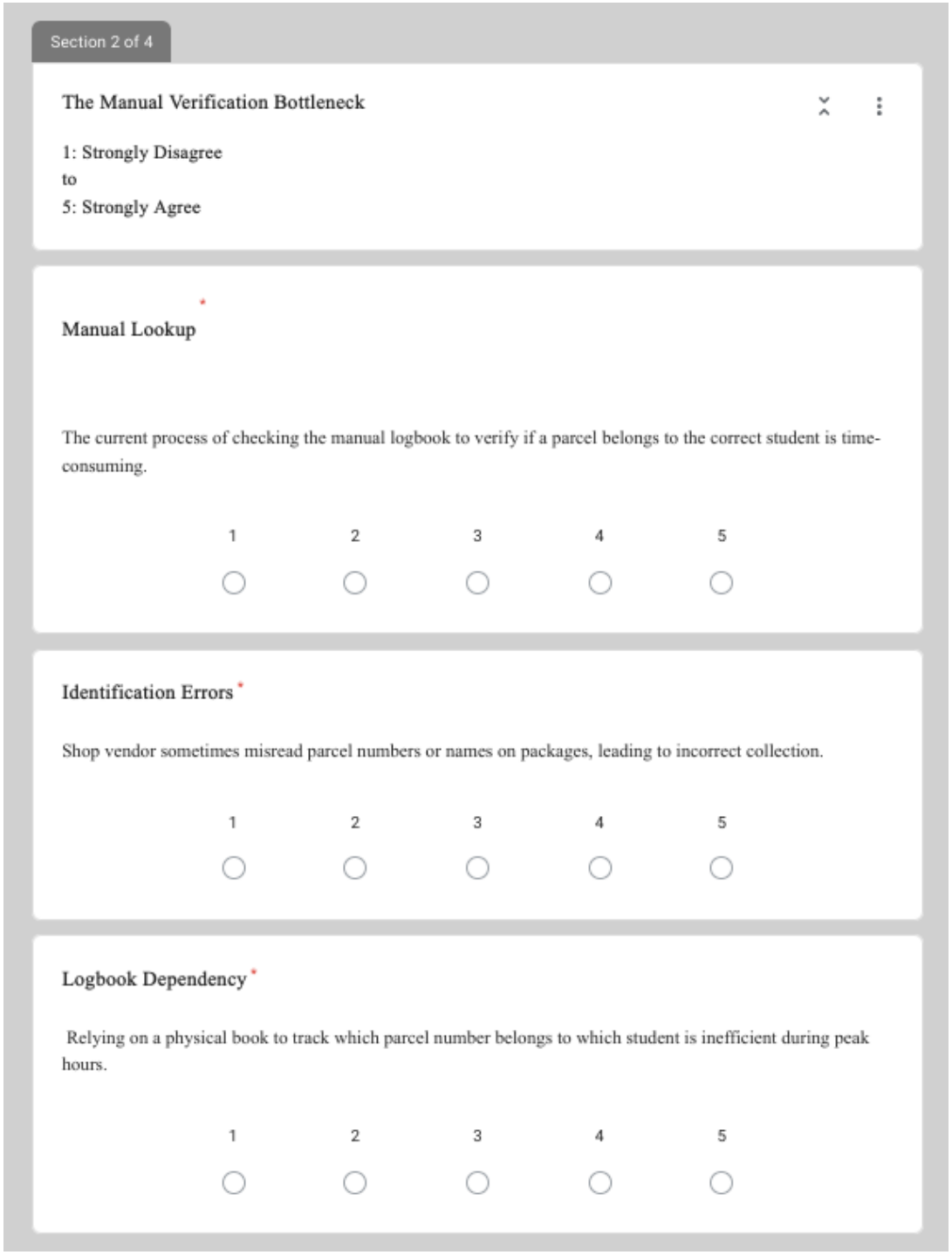


Figure A.1 : Questions for Existing System Workflow

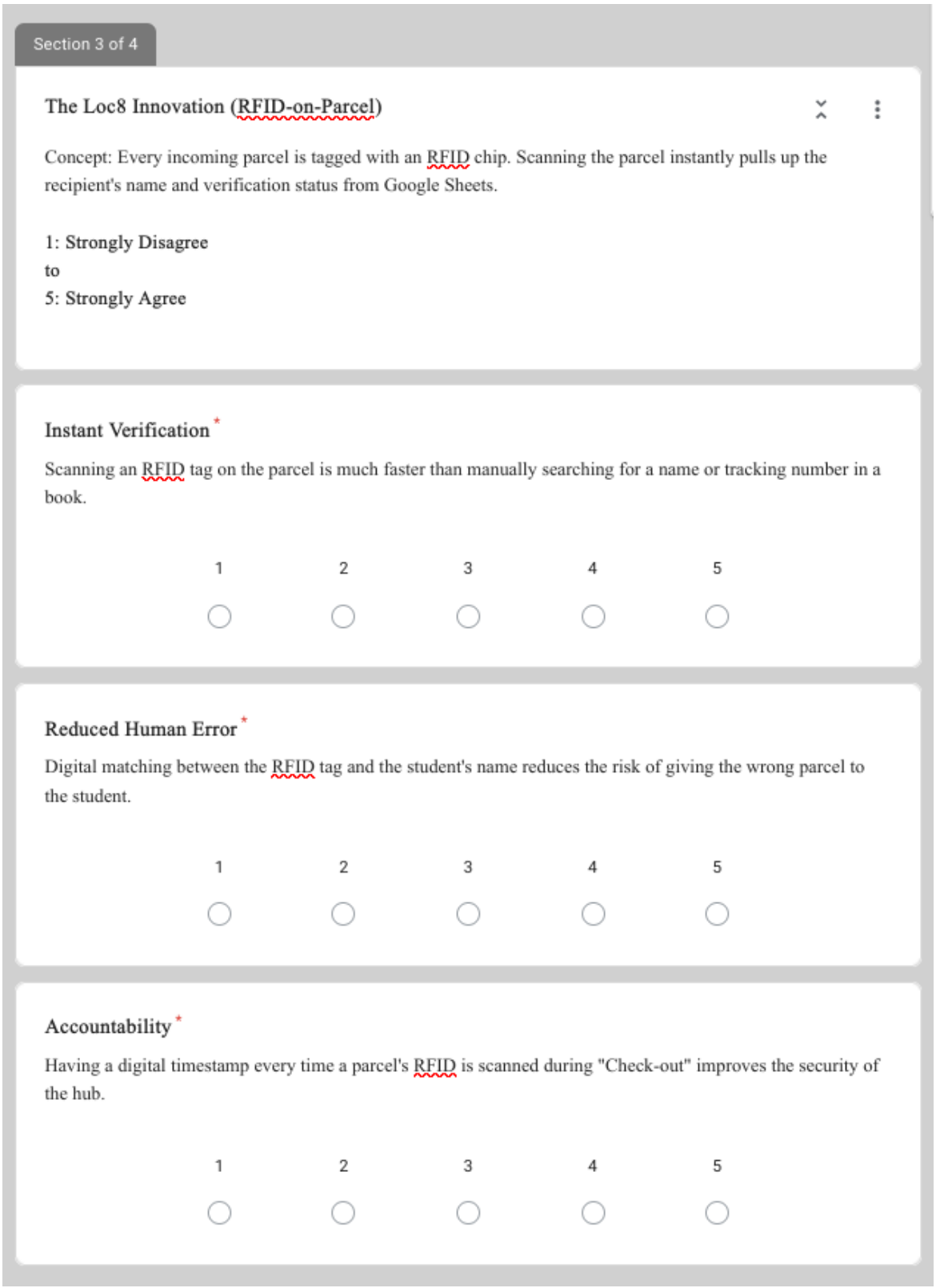


Figure A.2: Questions for Proposed System Workflow

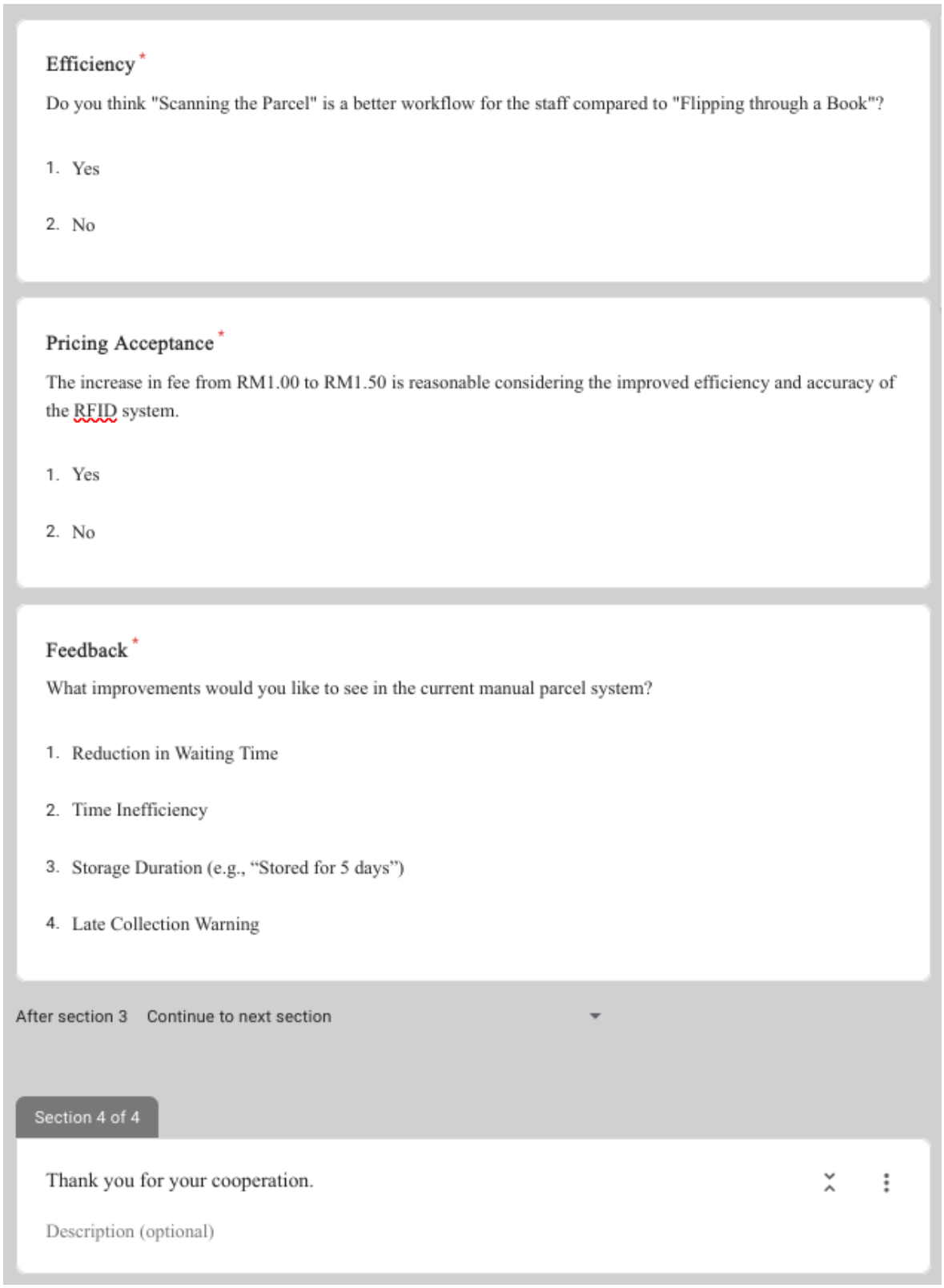


Figure A.3: Questions for User Feedback

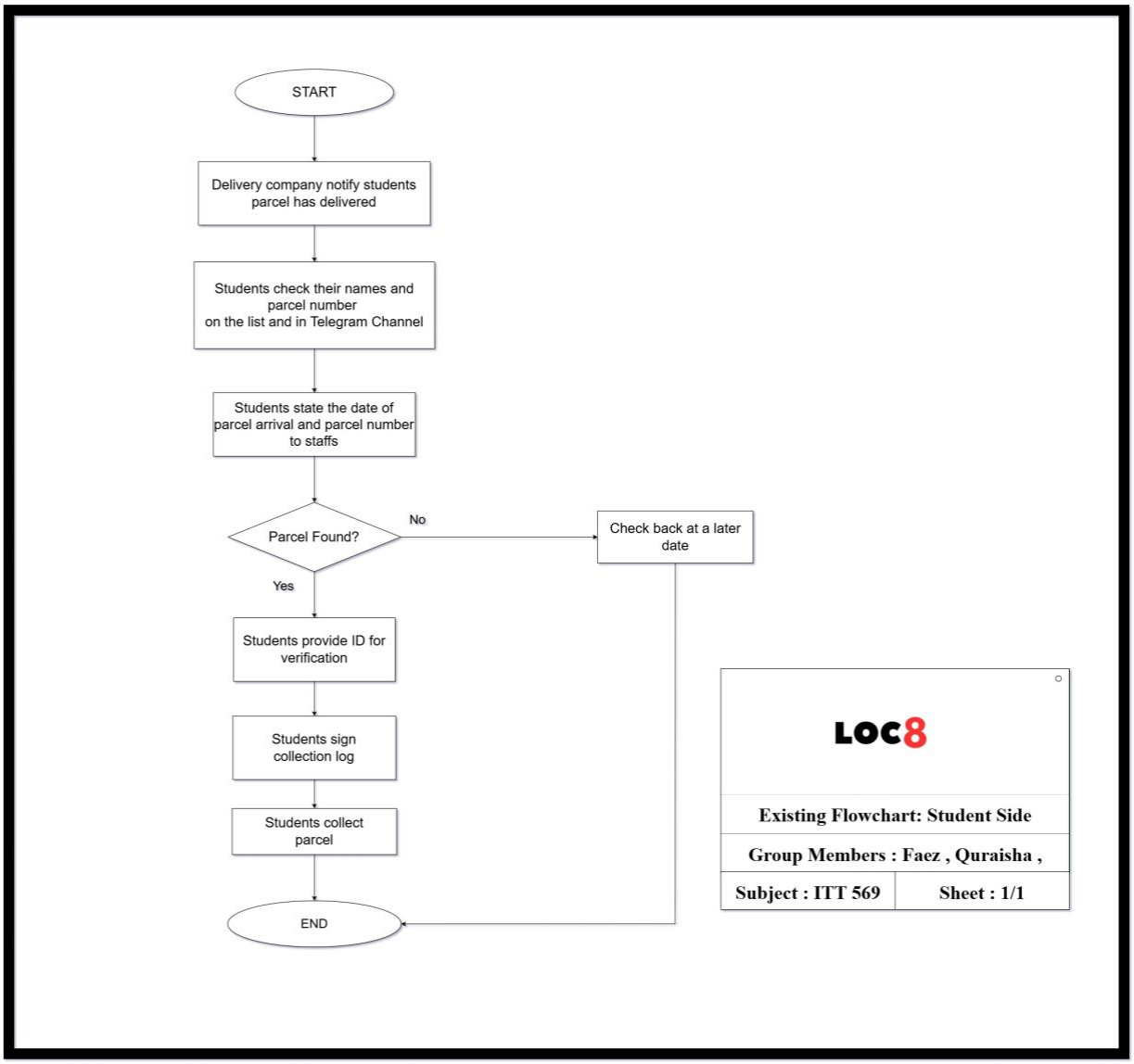


Figure A.4 : Flowchart of current Manual Parcel Management System on Student Side

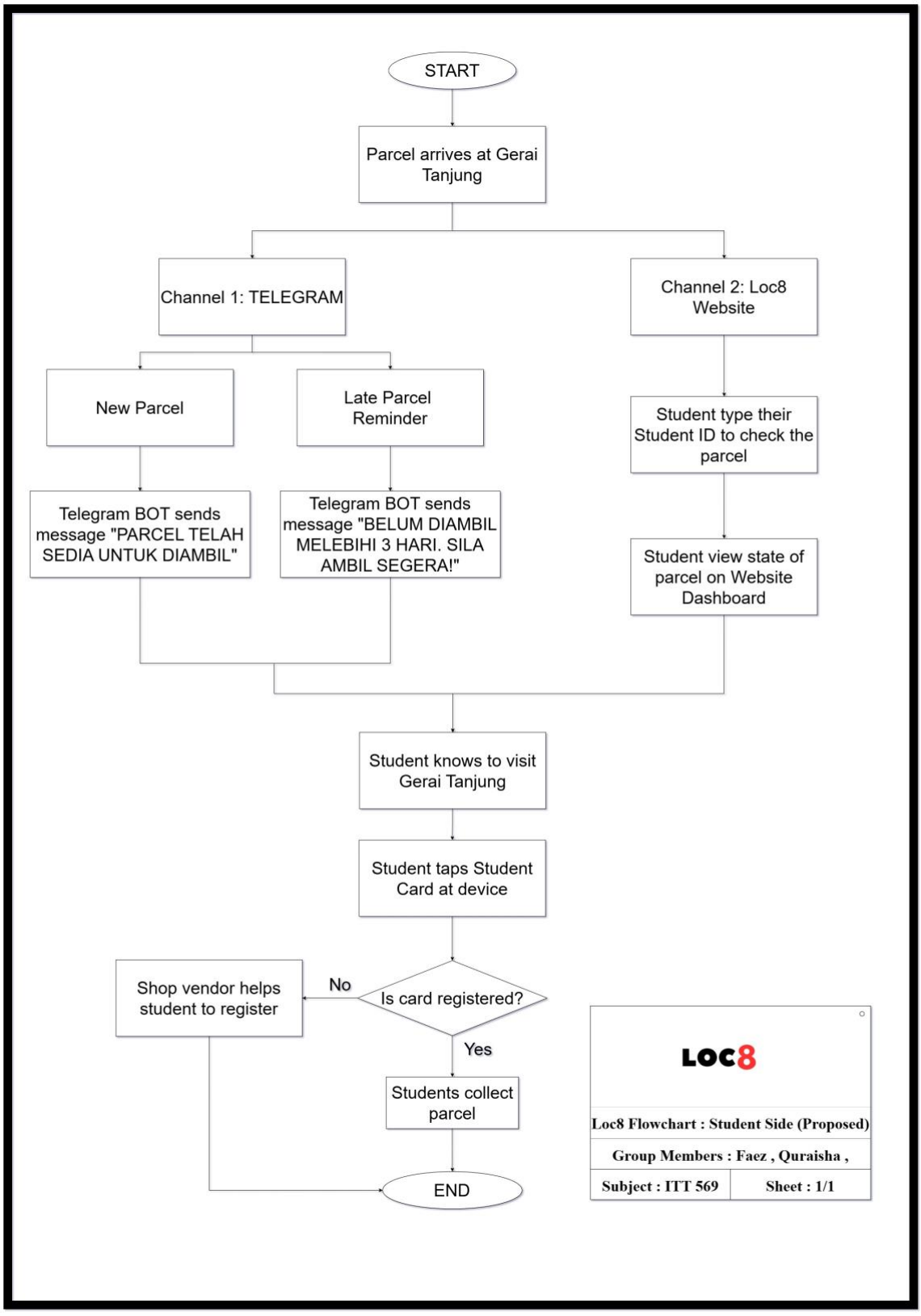


Figure A.5 : Flowchart of proposed Parcel Management System on Student Side

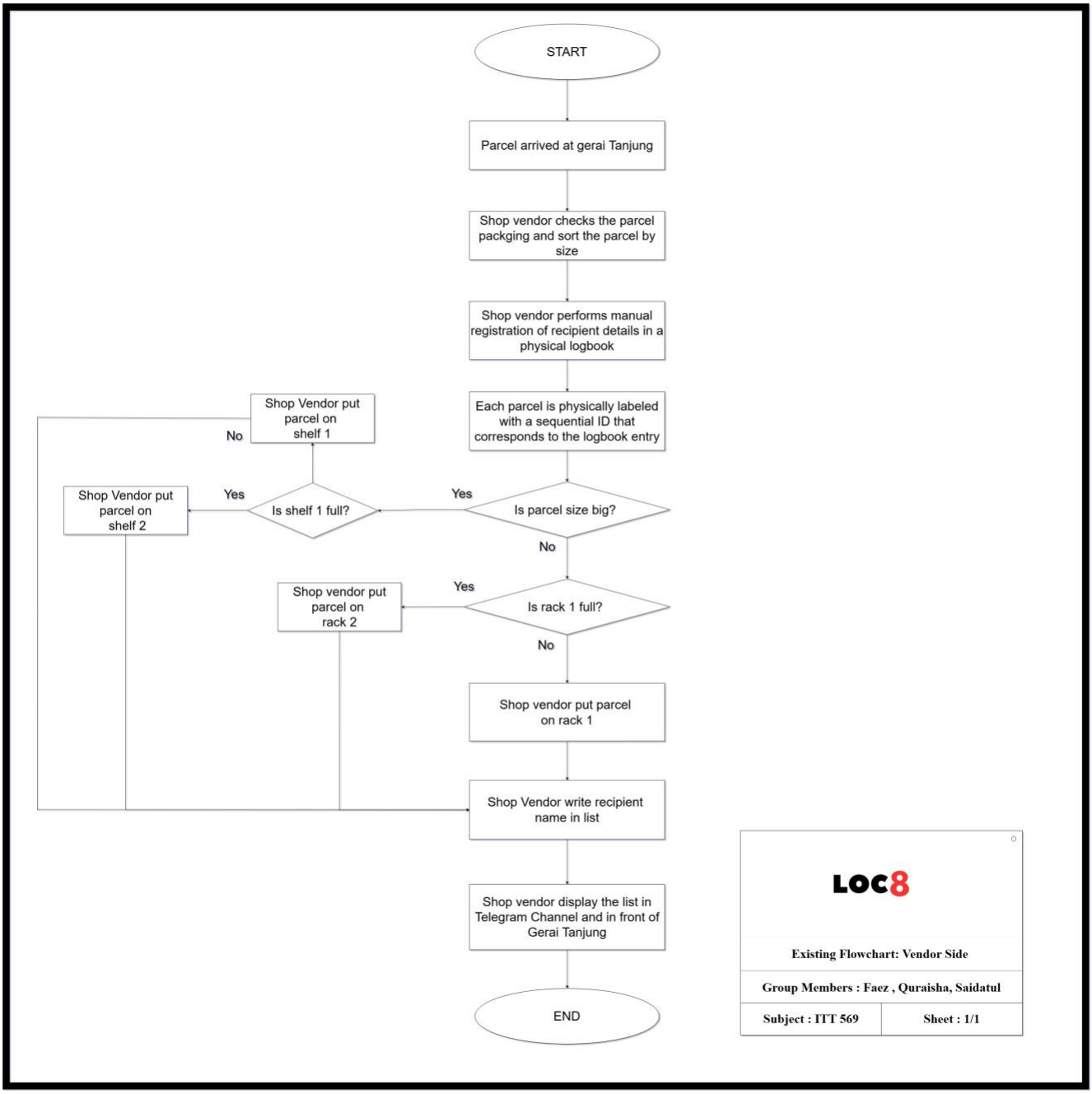


Figure A.6 : Flowchart of current Manual Parcel Management System

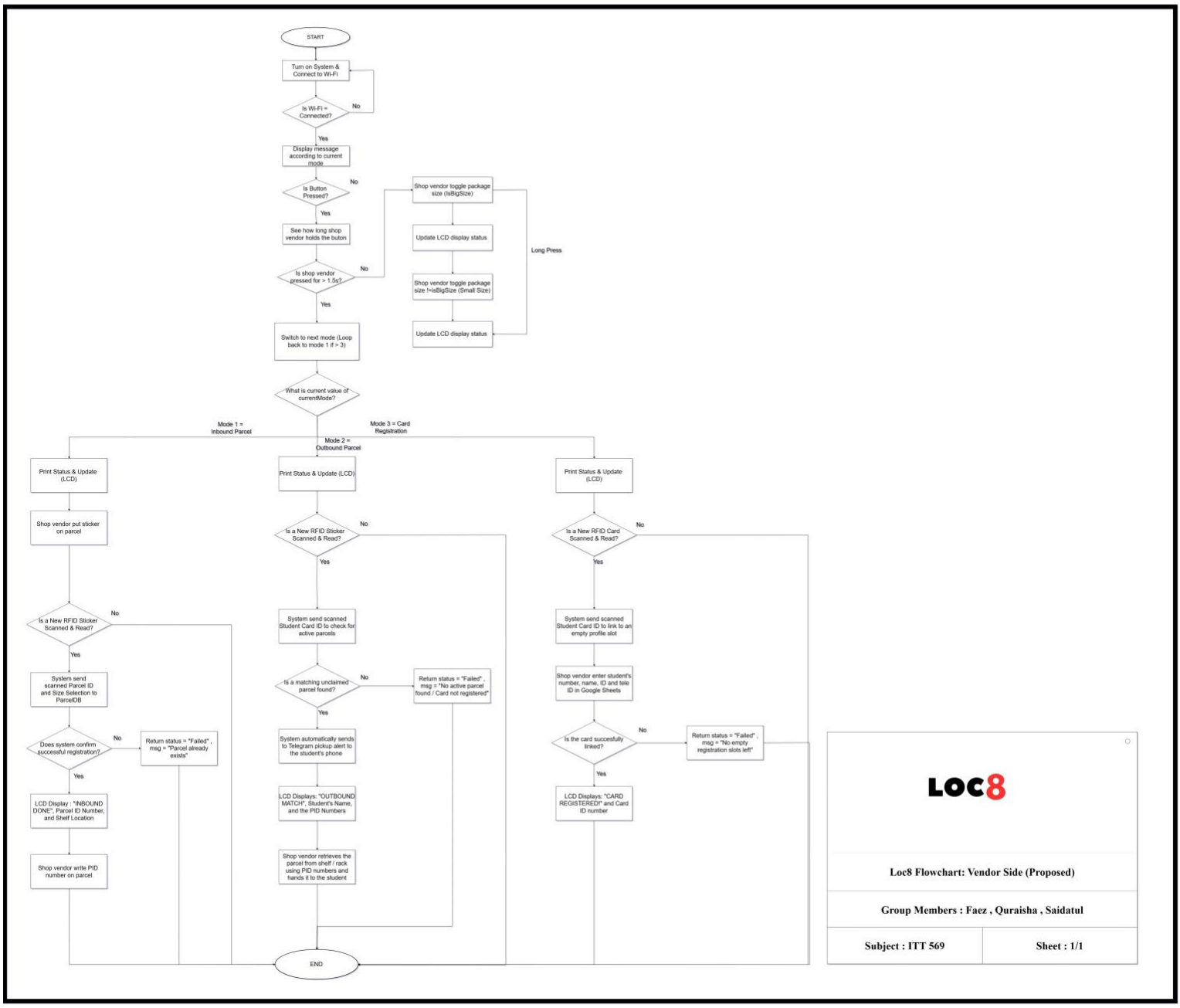


Figure A.7 : Flowchart of the IoT -Based Vandor Parcel Process

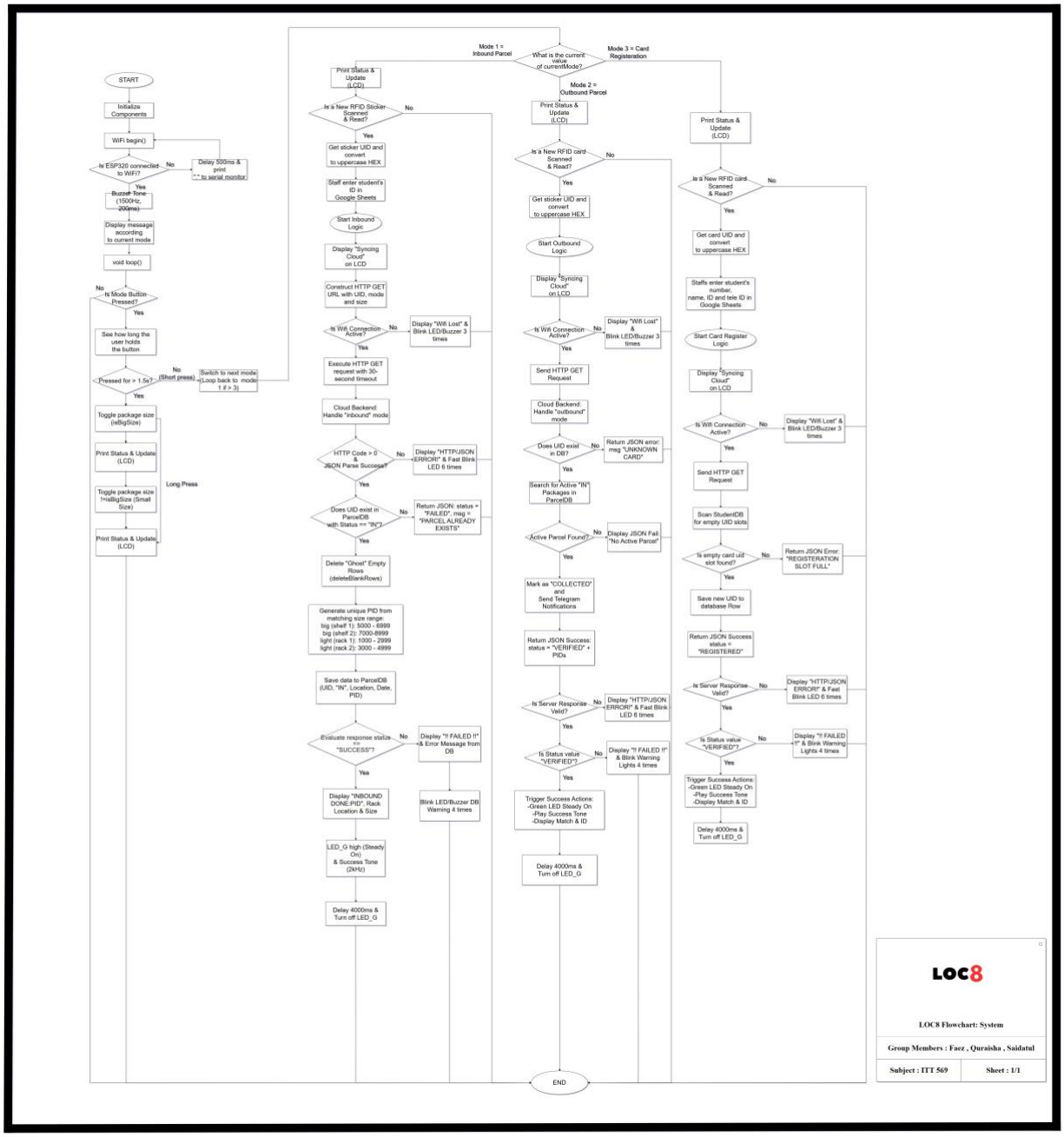


Figure A.8 : Flowchart of the Full System

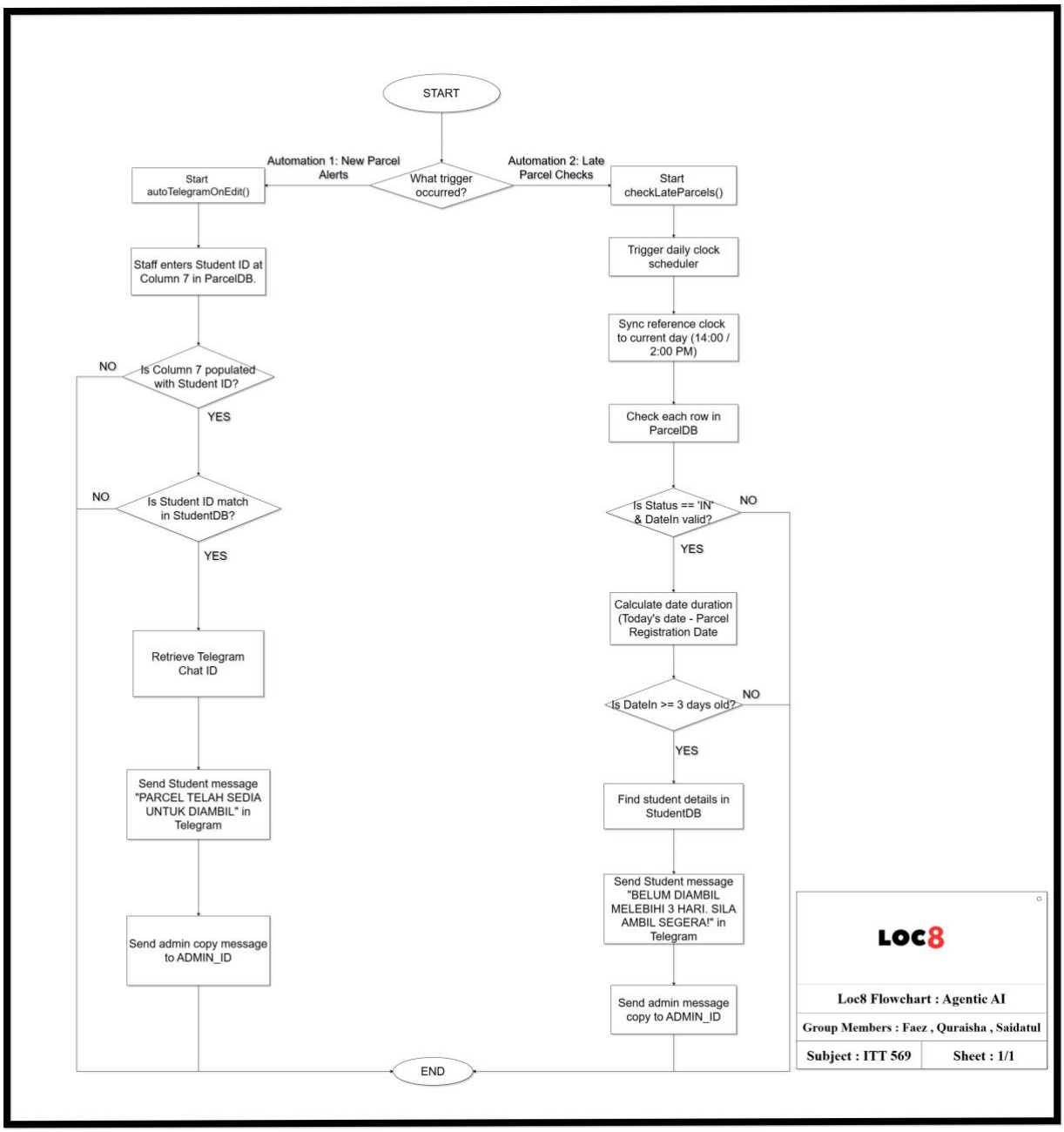


Figure A.9: Flowchart of Agentic AI

APPENDIX B :SOURCE CODE

Main.ino (ESP-32)

```
#include <SPI.h>
#include <MFRC522.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <WiFi.h>
#include <HTTPClient.h>
#include <WiFiClientSecure.h>
#include <ArduinoJson.h>

// --- KONFIGURASI WIFI & CLOUD ---
const char* ssid = "Faez";
const char* password = "faezyahizan123";
const char* scriptURL =
"https://script.google.com/macros/s/AKfycbyScMo4EWdoAGPeRGNSzdHe9GG2R
u4HINtdnQITiw5KO9CnuvgIty-c3dSVVsAOzzrU/exec";

// --- PIN MAPPING ---
#define SS_PIN 5
#define RST_PIN 4
#define LED_G 26
#define BUZZER 25
#define MODE_SW 14

MFRC522 rfid(SS_PIN, RST_PIN);
LiquidCrystal_I2C lcd(0x27, 20, 4);

int currentMode = 1;
bool isBigSize = false;

void setup() {
 Serial.begin(115200);
 delay(1000);
 Serial.println("\n============================================");
 Serial.println("[SYSTEM] STARTING LOC8 SMART PARCEL SYSTEM...");
 Serial.println("============================================");

 SPI.begin();
 rfid.PCD_Init();
```

```
 Wire.begin(21, 22);
 lcd.init();
 lcd.backlight();

 pinMode(LED_G, OUTPUT);
 pinMode(BUZZER, OUTPUT);
 pinMode(MODE_SW, INPUT_PULLUP);

 lcd.setCursor(0, 0);
 lcd.print("   LOC8 JASIN HUB   ");
 lcd.setCursor(0, 1);
 lcd.print("  CONNECTING WIFI   ");

 Serial.print("[WIFI] CONNECTING TO: ");
 Serial.println(ssid);
 WiFi.begin(ssid, password);
 while (WiFi.status() != WL_CONNECTED) {
   delay(500);
   Serial.print(".");
 }

 tone(BUZZER, 1500, 200);
 Serial.println("\n[WIFI] CONNECTED SUCCESSFULLY!");
 printCurrentModeToSerial();
 showIdleMessage();
}

void loop() {
 // Logik Butang Ditambah Baik (Single-Toggle on Hold)
 if (digitalRead(MODE_SW) == LOW) {
   Serial.println("\n[BUTTON] INPUT DETECTED! PROCESSING
DURATION...");
   unsigned long startTime = millis();
   bool longPressed = false;
   static bool actionExecuted = false;

   while (digitalRead(MODE_SW) == LOW) {
     if (millis() - startTime > 1500) {
       longPressed = true;

       if (!actionExecuted) {
         if (currentMode == 1) {
           isBigSize = !isBigSize;
           Serial.print("[BUTTON] LONG PRESS: TOGGLE SIZE CHANGED TO
-> ");
```

```
          Serial.println(isBigSize ? "BESAR (SHELF)" : "RINGAN
(RACK)");
          tone(BUZZER, 1200, 400);
        } else {
          Serial.println("[BUTTON] LONG PRESS: NO ACTION FOR THIS
MODE.");
          tone(BUZZER, 300, 200);
        }
        showIdleMessage();
        actionExecuted = true;
      }
    }
    delay(10);
  }

  if (!longPressed) {
    Serial.println("[BUTTON] SHORT PRESS: ADVANCING CORE SYSTEM
MODE...");
    currentMode++;
    if (currentMode > 3) {
      currentMode = 1;
    }
    tone(BUZZER, 1000, 100);
    printCurrentModeToSerial();
    showIdleMessage();
  }

  actionExecuted = false;
  delay(300);
 }

 // --- LOGIK SCANNER RFID ---
 if (!rfid.PICC_IsNewCardPresent() || !rfid.PICC_ReadCardSerial()) {
   return;
 }

 String scannedUID = "";
 for (byte i = 0; i < rfid.uid.size; i++) {
   scannedUID += String(rfid.uid.uidByte[i] < 0x10 ? "0" : "");
   scannedUID += String(rfid.uid.uidByte[i], HEX);
 }
 scannedUID.trim();
 scannedUID.toUpperCase();
 scannedUID.replace("-", "");
```

```
 // Memastikan sebarang aksara abjad pada UID ditukar ke huruf besar
 scannedUID.toUpperCase();

 Serial.println("\n-----------------------------------------");
 Serial.print("[RFID] TARGET CARD DETECTED!\n");
 Serial.print("[RFID] CLEAN UID STRING: ");
 Serial.println(scannedUID);

 if (currentMode == 1) {
   Serial.println("[SYSTEM] DIRECTING TO API: PARCEL INBOUND");
   sendToSheets(scannedUID, "inbound", isBigSize ? "big" : "light");
 } else if (currentMode == 2) {
   Serial.println("[SYSTEM] DIRECTING TO API: PARCEL OUTBOUND");
   sendToSheets(scannedUID, "outbound", "");
 } else if (currentMode == 3) {
   Serial.println("[SYSTEM] DIRECTING TO API: CARD REGISTRATION");
   sendToSheets(scannedUID, "register_card", "");
 }

 delay(3000);
 showIdleMessage();
}

void printCurrentModeToSerial() {
 Serial.println("-----------------------------------------");
 Serial.print("[MODE UPDATE] SYSTEM STATUS: MODE ");
 Serial.println(currentMode);
 if (currentMode == 1) Serial.println(">> MOD 1: PARCEL INBOUND ");
 else if (currentMode == 2) Serial.println(">> MOD 2: PARCEL
OUTBOUND");
 else if (currentMode == 3) Serial.println(">> MOD 3: CARD
REGISTRATION");
 Serial.println("-----------------------------------------");
}

void sendToSheets(String uid, String action, String size) {
 if (WiFi.status() == WL_CONNECTED) {
   HTTPClient http;
   WiFiClientSecure client;
   client.setInsecure();

   String url = String(scriptURL) + "?uid=" + uid + "&mode=" + action
+ "&size=" + size;

   Serial.print("[HTTP] DISPATCHING GET REQUEST: ");
```

```
  Serial.println(url);

  lcd.clear();
  lcd.setCursor(0, 1);
  lcd.print("  SYNCING CLOUD...  ");

  http.begin(client, url.c_str());

  // 30 SAAT TIMEOUT
  http.setTimeout(30000);
  http.setFollowRedirects(HTTPC_FORCE_FOLLOW_REDIRECTS);
  int httpCode = http.GET();

  Serial.print("[HTTP] RESPONSE STATUS CODE: ");
  Serial.println(httpCode);

  if (httpCode > 0) {
    String payload = http.getString();
    Serial.println("[HTTP] RAW PAYLOAD DATA:");
    Serial.println(payload);

    DynamicJsonDocument doc(2048);
    DeserializationError error = deserializeJson(doc, payload);

    if (!error) {
      String status = doc["status"].as<String>();
      status.toUpperCase();

      if (status == "SUCCESS" || status == "VERIFIED" || status ==
"REGISTERED") {
        Serial.println("[CLOUD DB] OPERATION STATUS: SUCCESS");

        // 🟢 LED MENYALA NYATA APABILA BERJAYA
        digitalWrite(LED_G, HIGH);
        tone(BUZZER, 2000, 200);
        lcd.clear();

        if (action == "register_card") {
          lcd.setCursor(0, 0);
          lcd.print("CARD REGISTERED!");
          lcd.setCursor(0, 2);
          lcd.print("UID: " + uid);
          lcd.setCursor(0, 3);
          lcd.print("DB STATUS: LINKED");
        }
```

```
        else if (action == "outbound") {
          String studentName = doc["name"].as<String>();
          studentName.toUpperCase();

          String pids = doc["collected_pids"].as<String>();

          Serial.print("[CLOUD DB] STUDENT IDENTITY MATCH: ");
          Serial.println(studentName);
          Serial.print("[CLOUD DB] BULK CHECKOUT PIDs: ");
          Serial.println(pids);

          lcd.setCursor(0, 0);
          lcd.print("OUTBOUND MATCH!");
          lcd.setCursor(0, 1);
          lcd.print("NAME: " + studentName.substring(0, 14));

          lcd.setCursor(0, 2);
          if (pids.length() > 16) {
            lcd.print("PIDs:" + pids.substring(0, 15));
            lcd.setCursor(0, 3);
            lcd.print(pids.substring(15));
          } else {
            lcd.print("PARCEL NO: " + pids);
            lcd.setCursor(0, 3);
            lcd.print("STATUS: COLLECTED");
          }
        } else {
          String pid = doc["pid"].as<String>();
          String shelfLoc = doc["shelf"].as<String>();
          shelfLoc.toUpperCase();

          lcd.setCursor(0, 0);
          lcd.print("INBOUND DONE: " + pid);
          lcd.setCursor(0, 1);
          lcd.print("LOC : " + shelfLoc);
          lcd.setCursor(0, 2);
          lcd.print("SIZE: " + String(size == "big" ? "BESAR" :
"RINGAN"));
          lcd.setCursor(0, 3);
          lcd.print("TULIS NO PADA BOX!");
        }
        delay(4000);
        digitalWrite(LED_G, LOW);
      } else {
        lcd.clear();
```

```
        String msg = doc["msg"].as<String>();
        msg.toUpperCase();

        Serial.print("[CLOUD DB] REJECTED: ");
        Serial.println(msg != "" ? msg : "REJECTED BY DB");

        lcd.setCursor(0, 0);
        lcd.print("!! FAILED !!");
        lcd.setCursor(0, 2);
        lcd.print(msg != "" ? msg : "DATABASE ERROR");
        lcd.setCursor(0, 3);
        lcd.print("UID: " + uid);

        // 🔴 KELIPAN AMARAN LED APABILA REJECTED
        for (int i = 0; i < 4; i++) {
          digitalWrite(LED_G, HIGH);
          tone(BUZZER, 300, 250);
          delay(250);
          digitalWrite(LED_G, LOW);
          delay(250);
        }
      }
    } else {
      Serial.println("[JSON] FAILED TO PROCESS INCOMING PAYLOAD.");
      lcd.clear();
      lcd.print("JSON ERROR!");

      // 🔴 KELIPAN KECEMASAN KELAJUAN TINGGI (JSON ERR)
      for (int i = 0; i < 6; i++) {
        digitalWrite(LED_G, HIGH);
        tone(BUZZER, 400, 150);
        delay(150);
        digitalWrite(LED_G, LOW);
        delay(150);
      }
    }
  } else {
    Serial.print("[HTTP] CLOUD CONNECTION FAILED. ERROR CODE: ");
    Serial.println(http.errorToString(httpCode).c_str());
    lcd.clear();
    lcd.setCursor(0, 1);
    lcd.print("    HTTP ERROR!    ");
    lcd.setCursor(0, 2);
    lcd.print("CODE: " + String(httpCode));
```

```
    // 🔴 KELIPAN KECEMASAN KELAJUAN TINGGI (HTTP TIMEOUT/ERR)
    for (int i = 0; i < 6; i++) {
      digitalWrite(LED_G, HIGH);
      tone(BUZZER, 400, 150);
      delay(150);
      digitalWrite(LED_G, LOW);
      delay(150);
    }
  }
  http.end();
 } else {
  Serial.println("[🚨 CRITICAL] WIFI DROPPED. ABORTING UPLOAD.");
  lcd.clear();
  lcd.print("WIFI LOST!");

  // 🔴 KELIPAN LED PERINGATAN WIFI TERPUTUS
  for (int i = 0; i < 3; i++) {
    digitalWrite(LED_G, HIGH);
    tone(BUZZER, 250, 400);
    delay(400);
    digitalWrite(LED_G, LOW);
    delay(200);
  }
 }
 Serial.println("-----------------------------------------------");
}

void showIdleMessage() {
 lcd.clear();
 lcd.setCursor(0, 0);
 lcd.print("--- LOC8 SYSTEM ---");
 switch (currentMode) {
  case 1:
    lcd.setCursor(0, 1);
    lcd.print("MOD: PARCEL INBOUND");
    lcd.setCursor(0, 2);
    lcd.print("SAIZ: ");
    lcd.print(isBigSize ? "BESAR (SHELF)" : "RINGAN (RACK)");
    lcd.setCursor(0, 3);
    lcd.print("HOLD SW: CHANGE SIZE");
    break;
  case 2:
    lcd.setCursor(0, 1);
```

```
    lcd.print("MOD: PARCEL OUTBOUND");
    lcd.setCursor(0, 2);
    lcd.print("READY FOR CHECKOUT");
    lcd.setCursor(0, 3);
    lcd.print("PRESS SW: CHANGE MODE");
    break;
  case 3:
    lcd.setCursor(0, 1);
    lcd.print("MOD: CARD REG");
    lcd.setCursor(0, 2);
    lcd.print("READY TO LINK CARD");
    lcd.setCursor(0, 3);
    lcd.print("PRESS SW: CHANGE MODE");
    break;
 }
}
```

Listing B.1 :Source Code for ESP32 Microcontroller

Google Apps Script in Sheets

```
// ============================================================
// MAIN CONFIG
// ============================================================
var TELEGRAM_TOKEN =
"8681848216:AAH6xJTOANF4Y3fETePTNTTo_nvt5oeWfJM";
var ADMIN_ID = "874249704";

// ============================================================
// 0. DARI EPS32
2)// ============================================================
function doGet(e) {
 var ss = SpreadsheetApp.getActiveSpreadsheet();
 var pSheet = ss.getSheetByName("ParcelDB");
 var sSheet = ss.getSheetByName("StudentDB");
  var uid = e.parameter.uid ?
e.parameter.uid.toString().trim().toUpperCase() : "";
 var mode = e.parameter.mode ?
e.parameter.mode.toString().trim().toLowerCase() : "";
 var size = e.parameter.size ?
e.parameter.size.toString().trim().toLowerCase() : "";
  if (uid === "") {
   return ContentService.createTextOutput(JSON.stringify({ "status":
"FAILED", "msg": "UID KOSONG"
})).setMimeType(ContentService.MimeType.JSON);
 }
  if (mode === "inbound") {
   deleteBlankRows();
 }
  var pData = pSheet.getDataRange().getValues();
 var sData = sSheet.getDataRange().getValues();
  // ------------------------------------------------------------
 // MOD 1: PARCEL INBOUND (Barang Masuk)
 // ------------------------------------------------------------
 if (mode === "inbound") {
   for (var i = 1; i < pData.length; i++) {
     var dbUID = pData[i][0] ?
pData[i][0].toString().trim().toUpperCase() : "";
     var dbStatus = pData[i][1] ?
pData[i][1].toString().trim().toUpperCase() : "";

     if (dbUID === uid && dbStatus === "IN") {
       return ContentService.createTextOutput(JSON.stringify({
```

```
        "status": "FAILED",
        "msg": "PARCEL DAH ADA"
      })).setMimeType(ContentService.MimeType.JSON);
    }
  }

  var finalLocation = "";
  var startID = 0;
  var endID = 0;

  var scriptProperties = PropertiesService.getScriptProperties();

  if (size === "big") {
    var lastShelf = scriptProperties.getProperty("LAST_SHELF") ||
"2";
    if (lastShelf === "2") {
      finalLocation = "SHELF 1"; startID = 5000; endID = 6999;
      scriptProperties.setProperty("LAST_SHELF", "1");
    } else {
      finalLocation = "SHELF 2"; startID = 7000; endID = 8999;
      scriptProperties.setProperty("LAST_SHELF", "2");
    }
  } else {
    var lastRack = scriptProperties.getProperty("LAST_RACK") || "2";
    if (lastRack === "2") {
      finalLocation = "RACK 1"; startID = 1000; endID = 2999;
      scriptProperties.setProperty("LAST_RACK", "1");
    } else {
      finalLocation = "RACK 2"; startID = 3000; endID = 4999;
      scriptProperties.setProperty("LAST_RACK", "2");
    }
  }

  var activeIDsSet = {};
  for (var i = 1; i < pData.length; i++) {
    var dbStatus = pData[i][1] ?
pData[i][1].toString().trim().toUpperCase() : "";
    var dbLoc = pData[i][2] ?
pData[i][2].toString().trim().toUpperCase() : "";
    var dbPid = pData[i][5] ? parseInt(pData[i][5]) : 0;

    if (dbLoc === finalLocation && dbStatus === "IN") {
      if (dbPid > 0) {
        activeIDsSet[dbPid] = true;
      }
```

```
    }
  }

  var generatedPID = startID;
  for (var id = startID; id <= endID; id++) {
    if (!activeIDsSet[id]) {
      generatedPID = id;
      break;
    }
  }

  var todayDate = new Date();
  var lastRowClean = pSheet.getLastRow();
  var nextRow = lastRowClean + 1;

  if (lastRowClean === 1 && pSheet.getRange(1, 1).getValue() === "")
{
    nextRow = 2;
  }

  pSheet.getRange(nextRow, 1).setValue(uid);
  pSheet.getRange(nextRow, 2).setValue("IN");
  pSheet.getRange(nextRow, 3).setValue(finalLocation);
  pSheet.getRange(nextRow, 4).setValue(todayDate);
  pSheet.getRange(nextRow, 6).setValue(generatedPID.toString());

  SpreadsheetApp.flush();

  return ContentService.createTextOutput(JSON.stringify({
    "status": "SUCCESS",
    "pid": generatedPID.toString(),
    "shelf": finalLocation
  })).setMimeType(ContentService.MimeType.JSON);
 }
  // ------------------------------------------------------------
 // MOD 2: PARCEL OUTBOUND (BULK CHECKOUT)
 // ------------------------------------------------------------
 else if (mode === "outbound") {
  var studentName = "";
  var studentID = "";
  var studentChatID = "";

  // Cari maklumat pelajar berdasarkan Kad UID yang diimbas
  for (var k = 1; k < sData.length; k++) {
```

```
     var dbStudentCard = sData[k][2] ?
sData[k][2].toString().trim().toUpperCase() : "";
     if (dbStudentCard === uid) {
       studentName = sData[k][0] ?
sData[k][0].toString().toUpperCase() : "PELAJAR";
       studentID = sData[k][3] ?
sData[k][3].toString().trim().toUpperCase() : "";
       studentChatID = sData[k][1] ? sData[k][1].toString().trim() :
"";
       break;
     }
   }

   if (studentID === "") {
     return ContentService.createTextOutput(JSON.stringify({
"status": "FAILED", "msg": "KAD TIDAK DIKENALI"
})).setMimeType(ContentService.MimeType.JSON);
   }

   var collectedPIDs = [];
   var rowsToUpdate = [];

   for (var i = 1; i < pData.length; i++) {
     var dbStatus = pData[i][1] ?
pData[i][1].toString().trim().toUpperCase() : "";
     var dbCardUID = pData[i][4] ?
pData[i][4].toString().trim().toUpperCase() : "";
     var dbStudID = pData[i][6] ?
pData[i][6].toString().trim().toUpperCase() : "";

     if (dbStatus === "IN" && (dbCardUID === uid || dbStudID ===
studentID)) {
       rowsToUpdate.push(i + 1);
       var pID = pData[i][5] ? pData[i][5].toString() : "-";
       if (pID !== "-") {
         collectedPIDs.push(pID);
       }
     }
   }

   if (collectedPIDs.length > 0) {
     for (var r = 0; r < rowsToUpdate.length; r++) {
       pSheet.getRange(rowsToUpdate[r], 2).setValue("COLLECTED");
     }
     SpreadsheetApp.flush();
```

```
     var pidsString = collectedPIDs.join(",");

     triggerBulkAlert(studentChatID, studentName, studentID,
pidsString);

     return ContentService.createTextOutput(JSON.stringify({
       "status": "VERIFIED",
       "name": studentName,
       "collected_pids": pidsString
     })).setMimeType(ContentService.MimeType.JSON);
   } else {
     return ContentService.createTextOutput(JSON.stringify({
       "status": "FAILED",
       "msg": "TIADA PARCEL AKTIF"
     })).setMimeType(ContentService.MimeType.JSON);
   }
 }
  // ------------------------------------------------------------
 // MOD 3: CARD REGISTRATION
 // ------------------------------------------------------------
 else if (mode === "register_card") {
   var registered = false;
   for (var j = 1; j < sData.length; j++) {
     var currentUID = sData[j][2] ? sData[j][2].toString().trim() :
"";
     if (currentUID === "" || currentUID === "-") {
       sSheet.getRange(j + 1, 3).setValue(uid);
       registered = true;
       break;
     }
   }

   SpreadsheetApp.flush();

   if (registered) {
     return ContentService.createTextOutput(JSON.stringify({
"status": "REGISTERED" })).setMimeType(ContentService.MimeType.JSON);
   } else {
     return ContentService.createTextOutput(JSON.stringify({
"status": "FAILED", "msg": "SLOT REGISTER PENUH"
})).setMimeType(ContentService.MimeType.JSON);
   }
 }
```

```
  return ContentService.createTextOutput(JSON.stringify({ "status":
"FAILED", "msg": "MOD TIDAK SAH"
})).setMimeType(ContentService.MimeType.JSON);
}

// ============================================================
// 1. ON-EDIT FUNC
// ============================================================
function autoTelegramOnEdit(e) {
 if (!e) return;
 var range = e.range;
 var sheet = range.getSheet();
 if (sheet.getName() !== "ParcelDB") return;
 var row = range.getRow();
 var col = range.getColumn();
  if (col === 7) {
   var studID = e.value ? e.value.toString().trim().toUpperCase() :
"";
   if (studID !== "" && studID !== "-") {
     sheet.getRange(row, 2).setValue("IN");
     Utilities.sleep(1500);
     SpreadsheetApp.flush();
     triggerSingleAlert(row, "Kemaskini");
   }
 }
}

// ============================================================
// 2. CHECK LATE PARCEL FUNC
// ============================================================
function checkLateParcels() {
 var ss = SpreadsheetApp.getActiveSpreadsheet();
 var pSheet = ss.getSheetByName("ParcelDB");
 var pData = pSheet.getDataRange().getValues();
 var today = new Date();
 today.setHours(0,0,0,0);

 for (var i = 1; i < pData.length; i++) {
   var status = pData[i][1];
   var dateInVal = pData[i][3];

   if (status == "IN" && dateInVal && dateInVal instanceof Date) {
     var dateIn = new Date(dateInVal);
     dateIn.setHours(0,0,0,0);
     var diffTime = today.getTime() - dateIn.getTime();
```

```
     var diffDays = Math.floor(diffTime / (1000 * 60 * 60 * 24));

     if (diffDays >= 3) {
       triggerSingleAlert(i + 1, "Lewat");
     }
   }
 }
}

// ============================================================
// 3. TELE NOTI (SINGLE ITEM)
// ============================================================
function triggerSingleAlert(row, type) {
 var ss = SpreadsheetApp.getActiveSpreadsheet();
 var pSheet = ss.getSheetByName("ParcelDB");
 var sSheet = ss.getSheetByName("StudentDB");
  var studID = pSheet.getRange(row, 7).getValue();
 studID = studID ? studID.toString().trim().toUpperCase() : "";
  var parcelNo = pSheet.getRange(row, 6).getValue();
 parcelNo = (parcelNo !== null && parcelNo !== "") ?
parcelNo.toString() : "-";
  if (studID == "" || studID == "-") return;

 var sData = sSheet.getDataRange().getValues();

 for (var j = 1; j < sData.length; j++) {
   var dbStudID = sData[j][3] ?
sData[j][3].toString().trim().toUpperCase() : "";
   if (dbStudID === studID) {
     var chatID = sData[j][1];
     var name = sData[j][0] ? sData[j][0].toString().toUpperCase() :
"PELAJAR";

     if (chatID && chatID.toString().length > 5) {
       var note = "";
       if (type == "Collected") note = "PARCEL TELAH BERJAYA DIAMBIL.
TERIMA KASIH!";
       else if (type == "Lewat") note = "BELUM DIAMBIL MELEBIHI 3
HARI. SILA AMBIL SEGERA!";
       else note = "PARCEL TELAH SEDIA UNTUK DIAMBIL.";

       sendTelegramAlert(chatID, name, studID, parcelNo, note);
     }
     break;
   }
```

```
 }
}

// ============================================================
// 4. TELE NOTI BULK PARCEL
// ============================================================
function triggerBulkAlert(chatID, name, studID, allPIDs) {
 if (!chatID || chatID.length <= 5) return;
  var note = "SEMUA PARCEL BERIKUT TELAH DIAMBIL SEMASA BULK
CHECKOUT. TERIMA KASIH!";
 sendTelegramAlert(chatID, name, studID, allPIDs, note);
}

// ============================================================
// 5. HELPER TELEGRAM
// ============================================================
function sendTelegramAlert(destID, name, studID, pID, info) {
 var message = "📦 *LOC8 NOTIFICATION* 📦\n\n" +
               "PENERIMA: *" + name + "*\n" +
               "ID PELAJAR: `" + studID + "`\n" +
               "PARCEL NO: [#" + pID + "]\n" +
               "STATUS: " + info + "\n\n" +
               "LOKASI: GERAI TANJUNG.";
  executeSend(destID, message);
 if (ADMIN_ID && destID != ADMIN_ID) {
   executeSend(ADMIN_ID, "📢 *ADMIN COPY*\n" + message);
 }
}

function executeSend(id, msg) {
 if (!id || id == "undefined" || id == "") return;
 try {
   var url = "https://api.telegram.org/bot" + TELEGRAM_TOKEN +
"/sendMessage?chat_id=" + id +
             "&text=" + encodeURIComponent(msg) +
"&parse_mode=Markdown";
   UrlFetchApp.fetch(url);
 } catch (err) {
   Logger.log("RALAT HANTAR KE " + id + ": " + err.toString());
 }
}

// ============================================================
// BUANG TABLE KOSONG DLM SHEET
```

```
// ============================================================
function deleteBlankRows() {
 var ss = SpreadsheetApp.getActiveSpreadsheet();
 var sheet = ss.getSheetByName("ParcelDB");
 var lastRow = sheet.getLastRow();
  if (lastRow <= 1) return;
  var range = sheet.getRange(1, 1, lastRow, 1);
 var values = range.getValues();
  for (var i = lastRow - 1; i >= 1; i--) {
   if (values[i][0] === "" || values[i][0] === null) {
     sheet.deleteRow(i + 1);
   }
 }
}
```

Listing B.2 :Source Code for Backend (Google Apps Scrip